Doubt Class with Lakshay

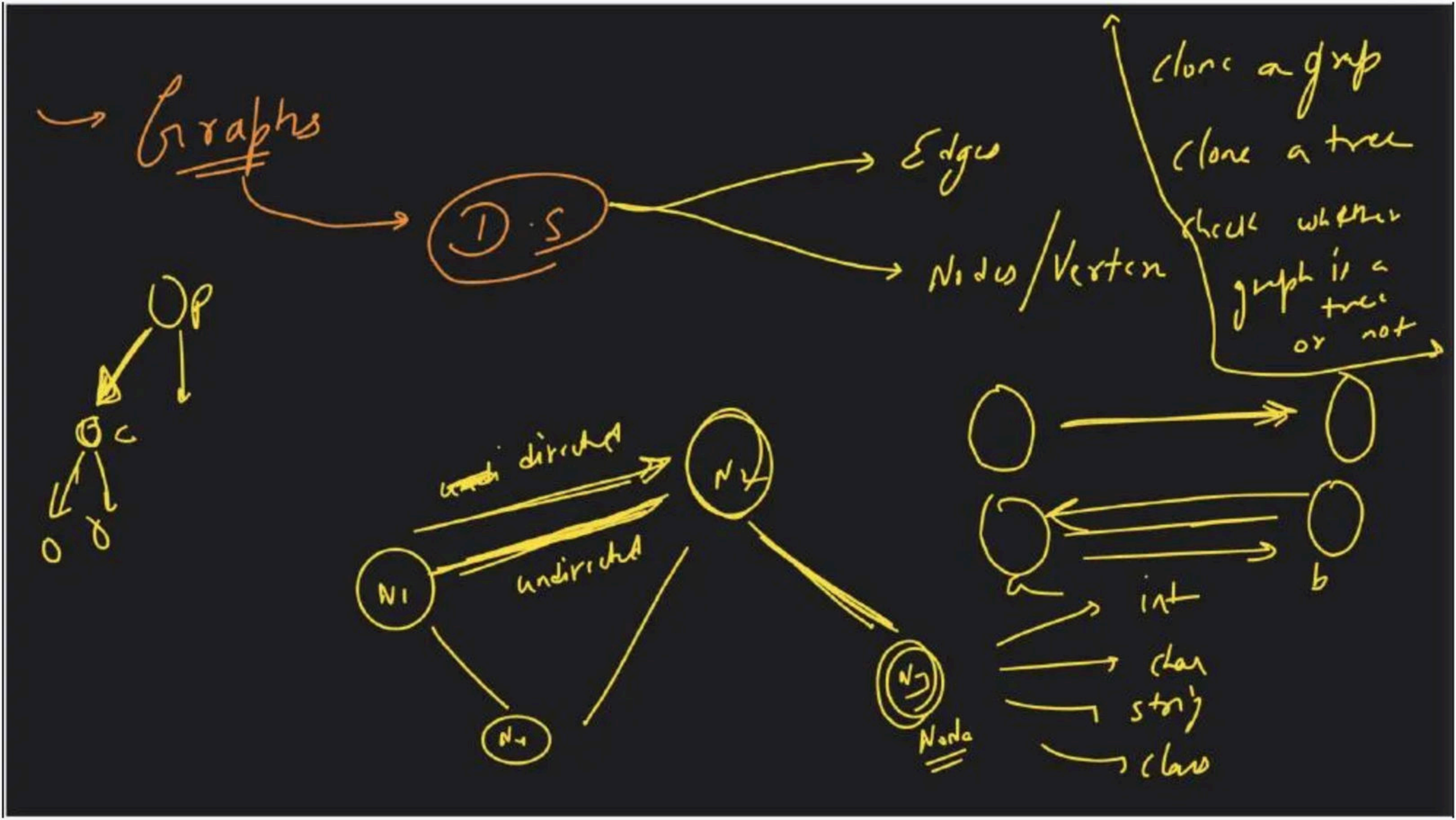
Special class

Love Babbar • Oct 13, 2024

Graph Class - 1

Special class

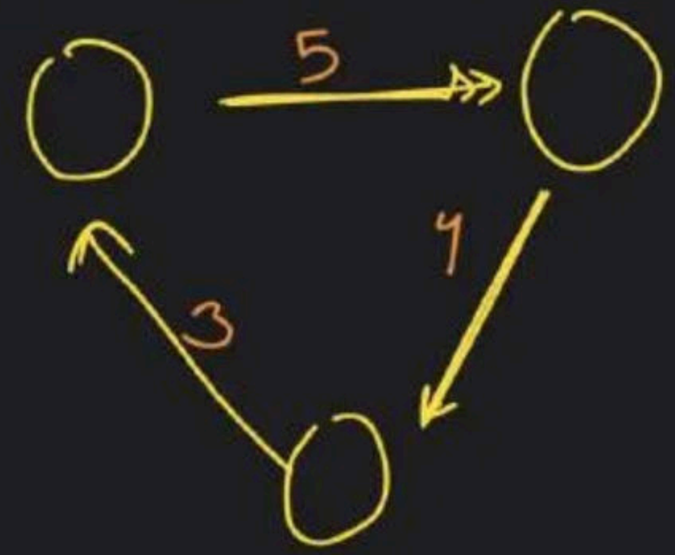
Love Babbar • Sept 25, 2024


Graphs
DS
Edges
Nodes/Vertex
clone a graph
clone a tree
check whether graph is a tree or not
P
C
directed
undirected
N1
N2
N3
N4
Node
int
char
string
class
a
b

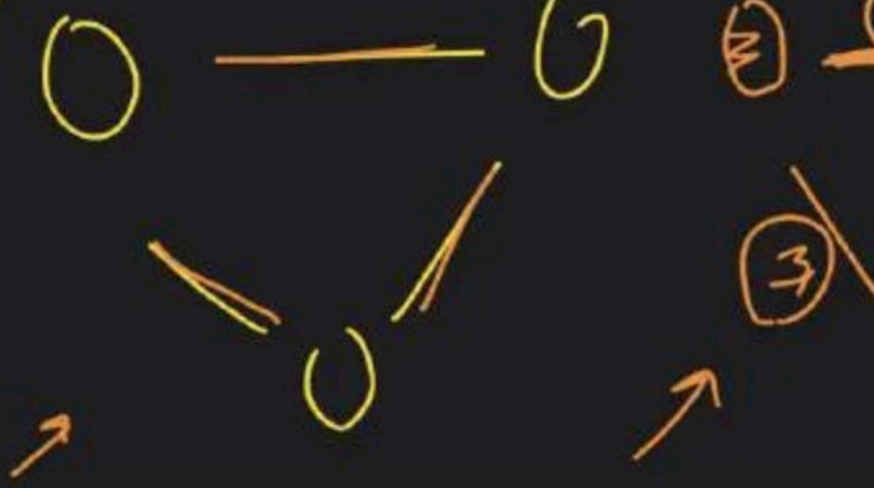
graph

- edge
  - directed
  - undirected
- node

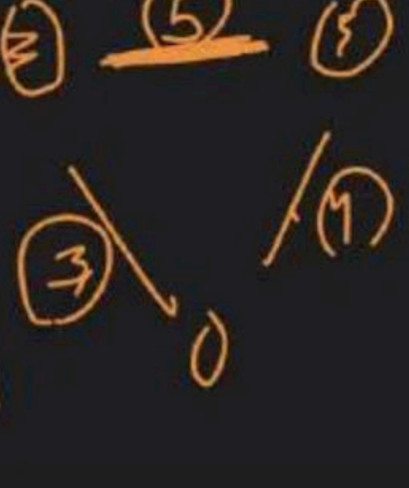
weighted Directed Graph



unweighted undirected graph



weighted undirected graph

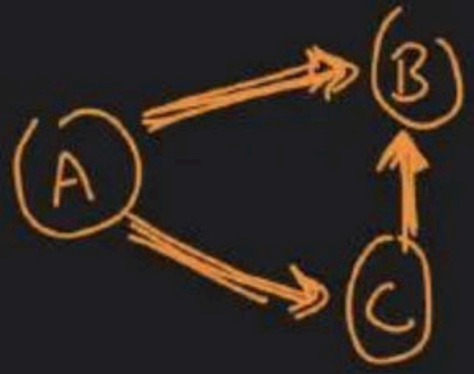


Degree

Indegree

Outdgre

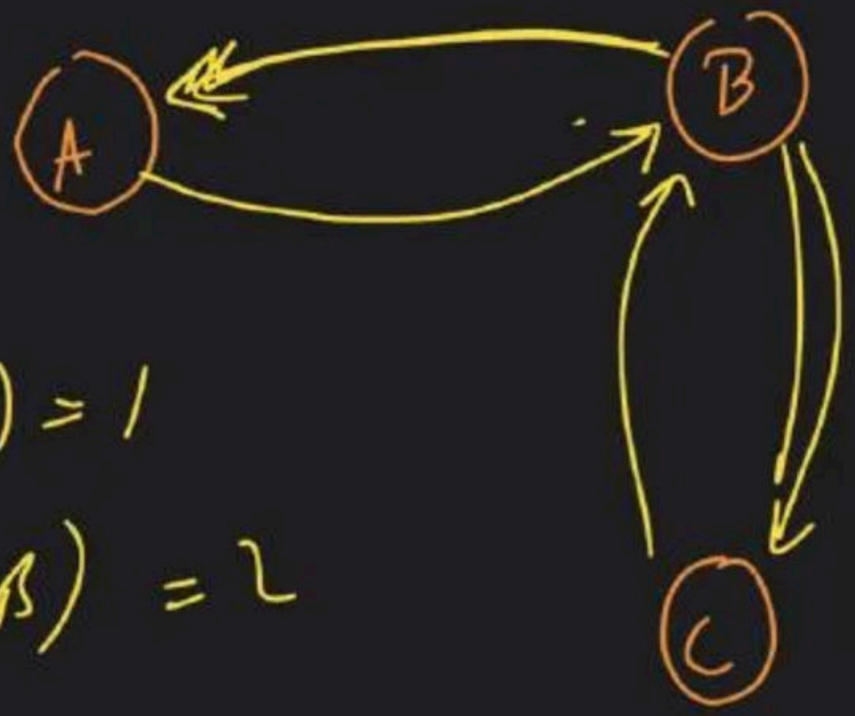
Directed



Indgree (A) = 0

Outdgree (A) = 2

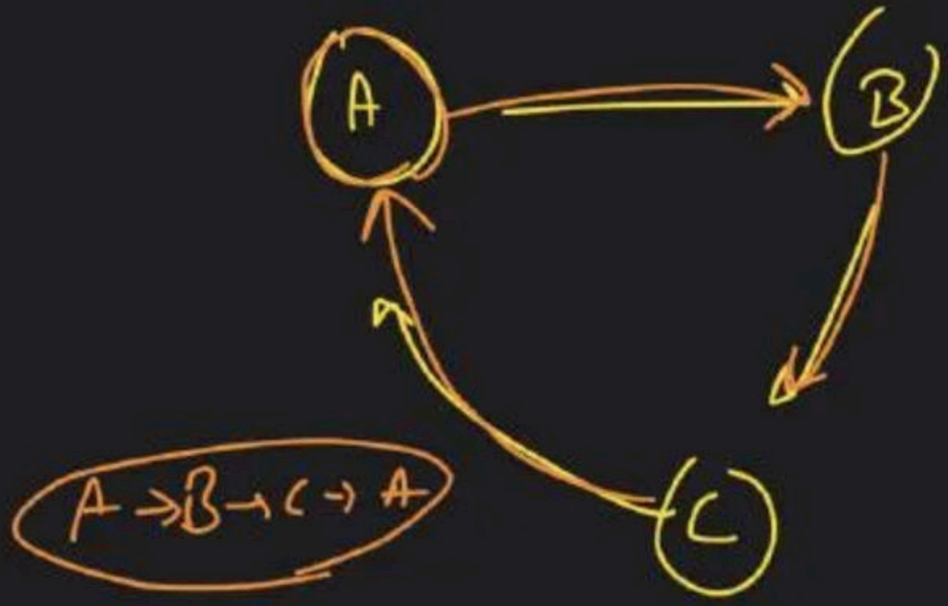
Indgree (B) = 2



indegree (A) = 1

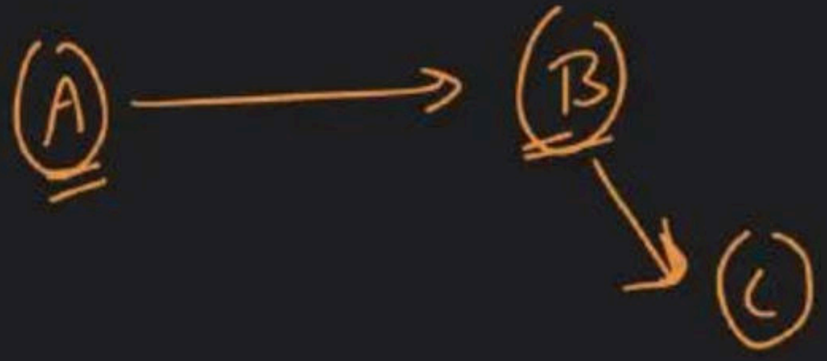
outdegree (B) = 2

Cycle



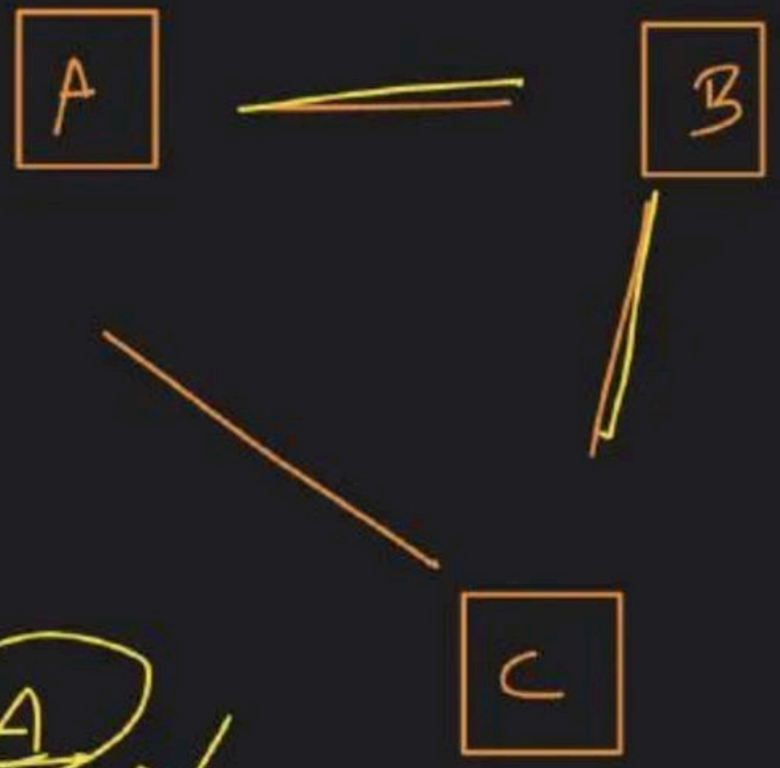
A → B → C → A

Cyclic graph



Acyclic graph

Path

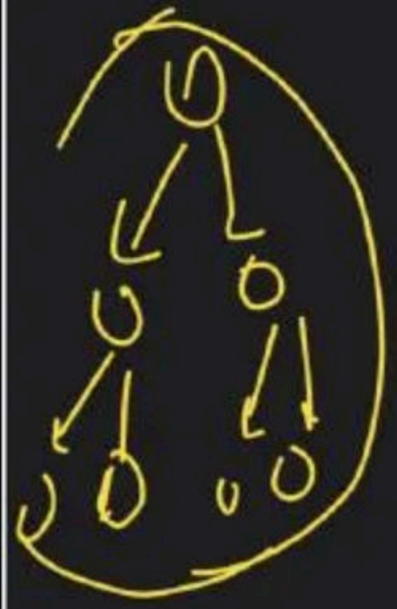


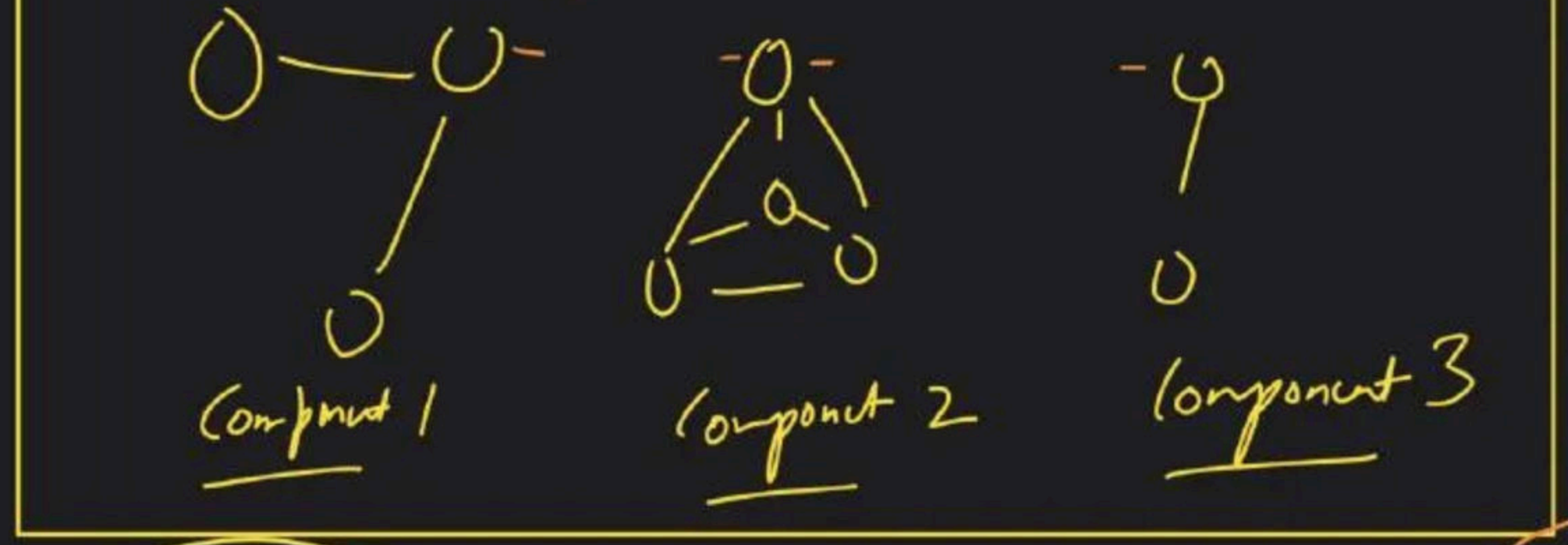
A - B - C

A - B - C - A

Components

Connected Component





Component 1

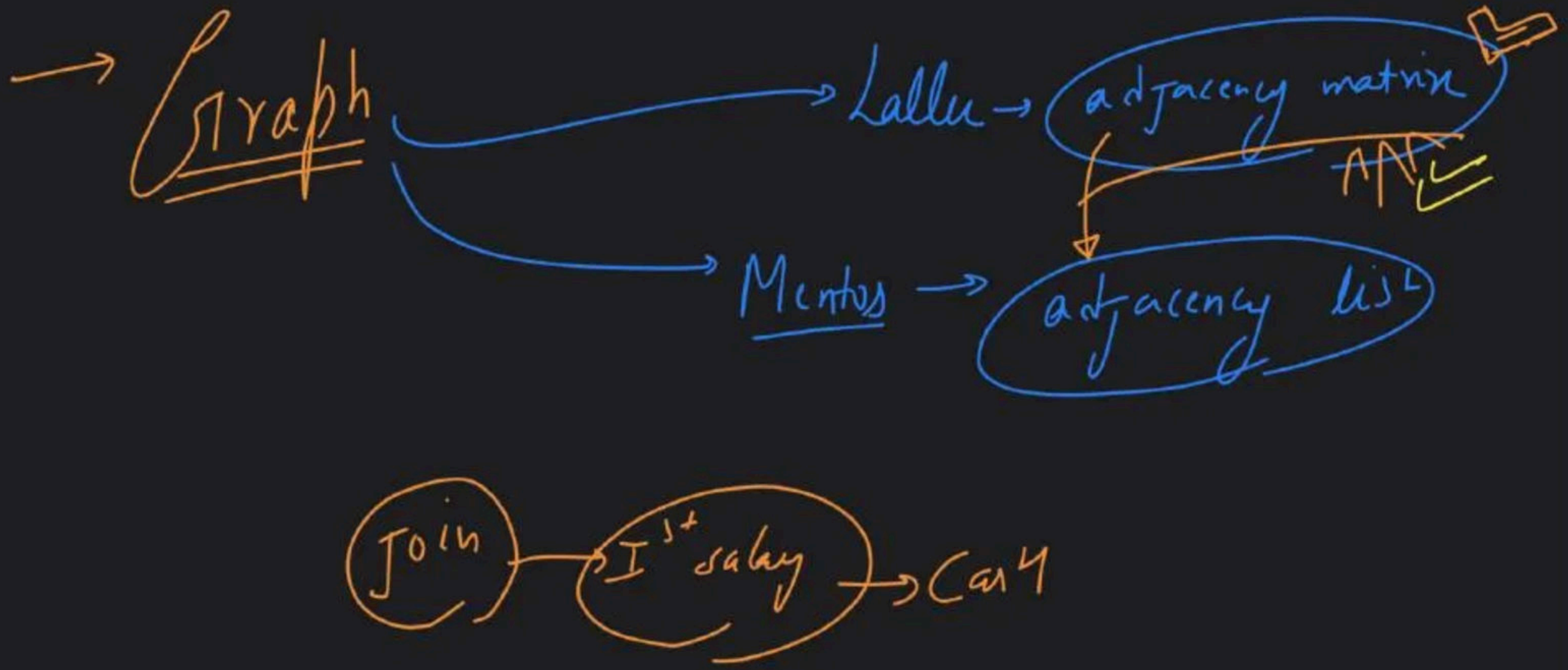
Component 2

Component 3

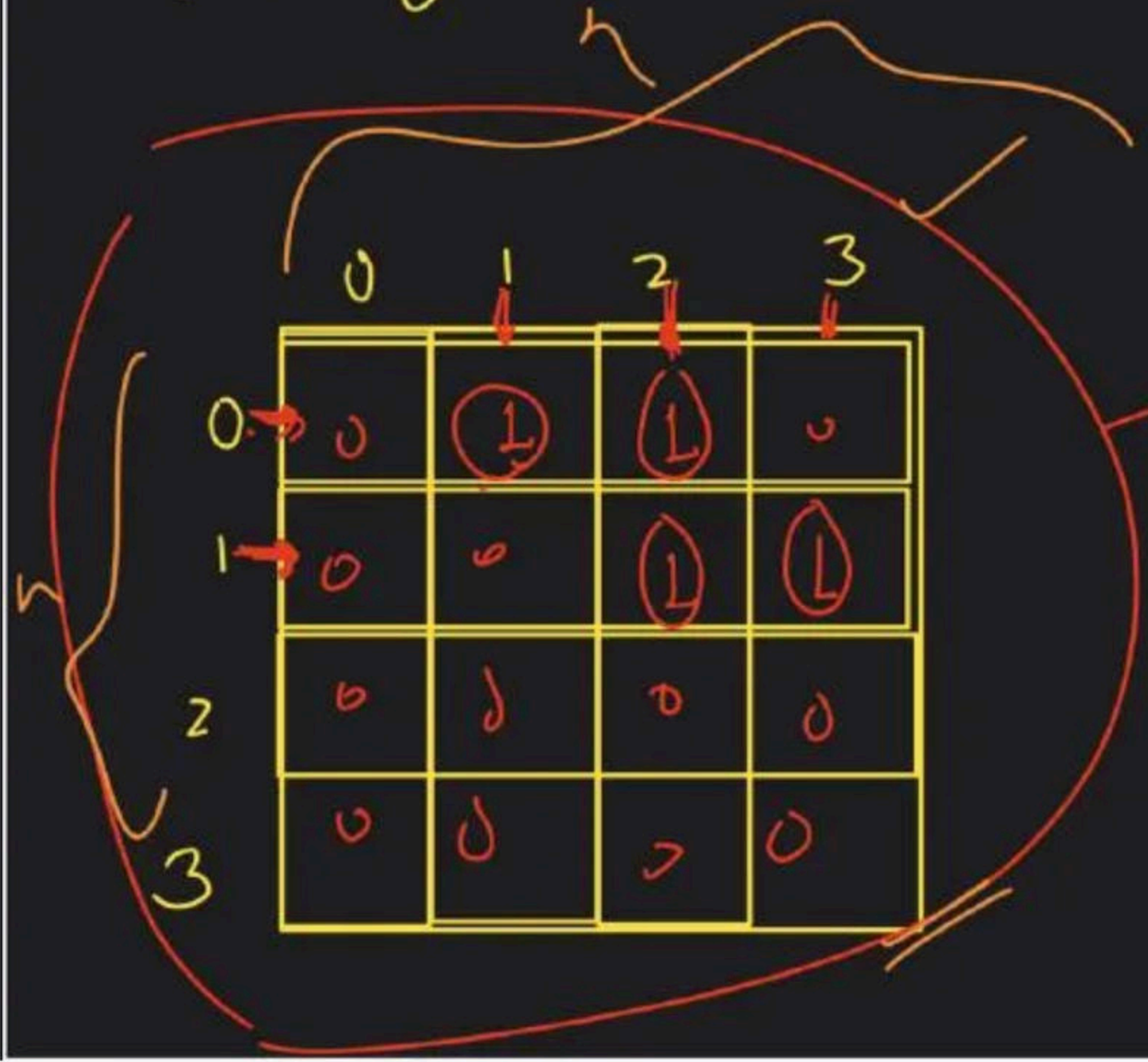
graph g

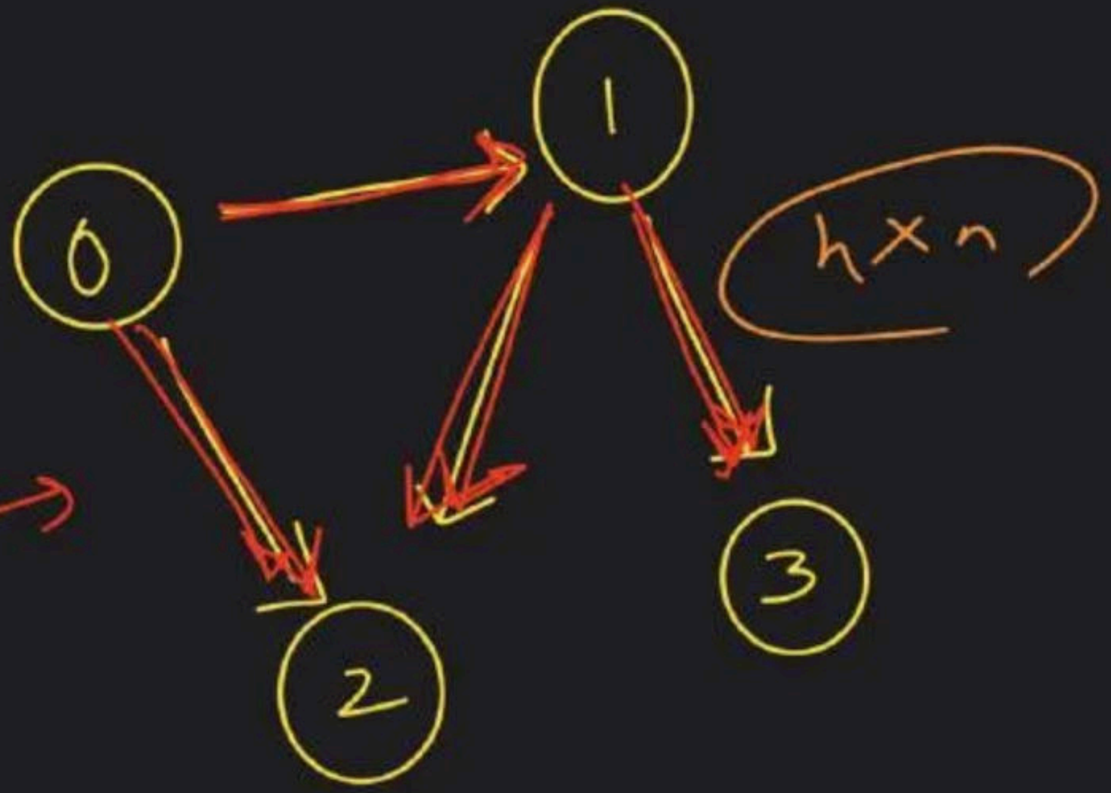
disconnected Components

forest


Graph
Lallu
adjacency matrix
Mantos
adjacency list
Join
1st salary
Cash

Adjacency matrix → 2d array

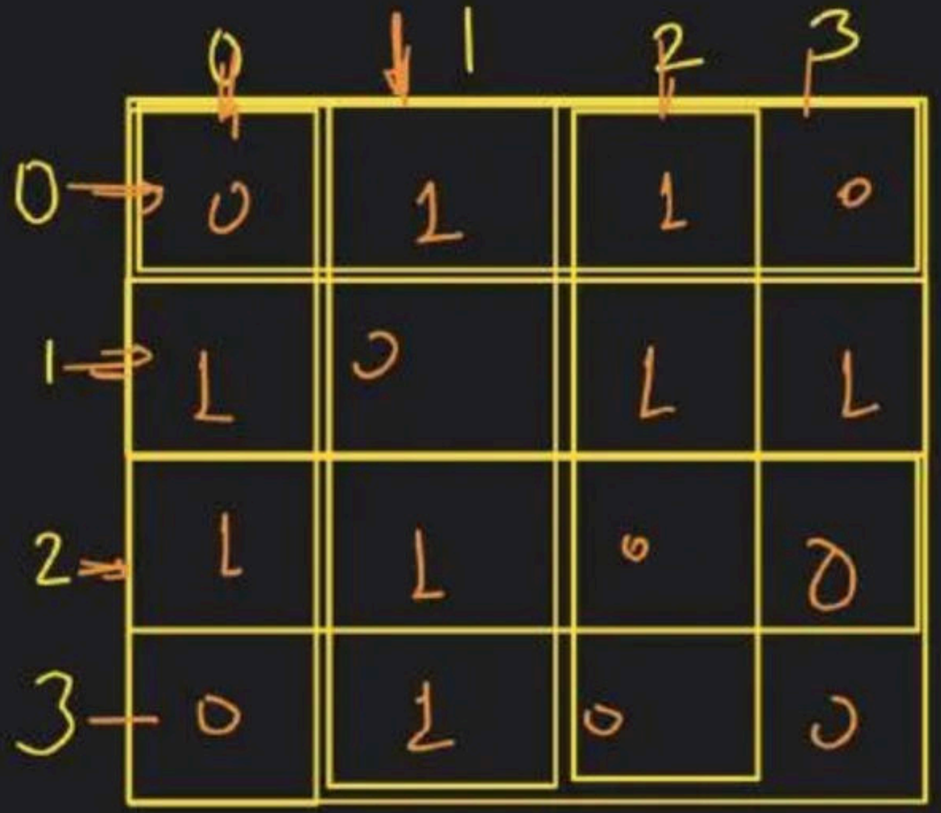


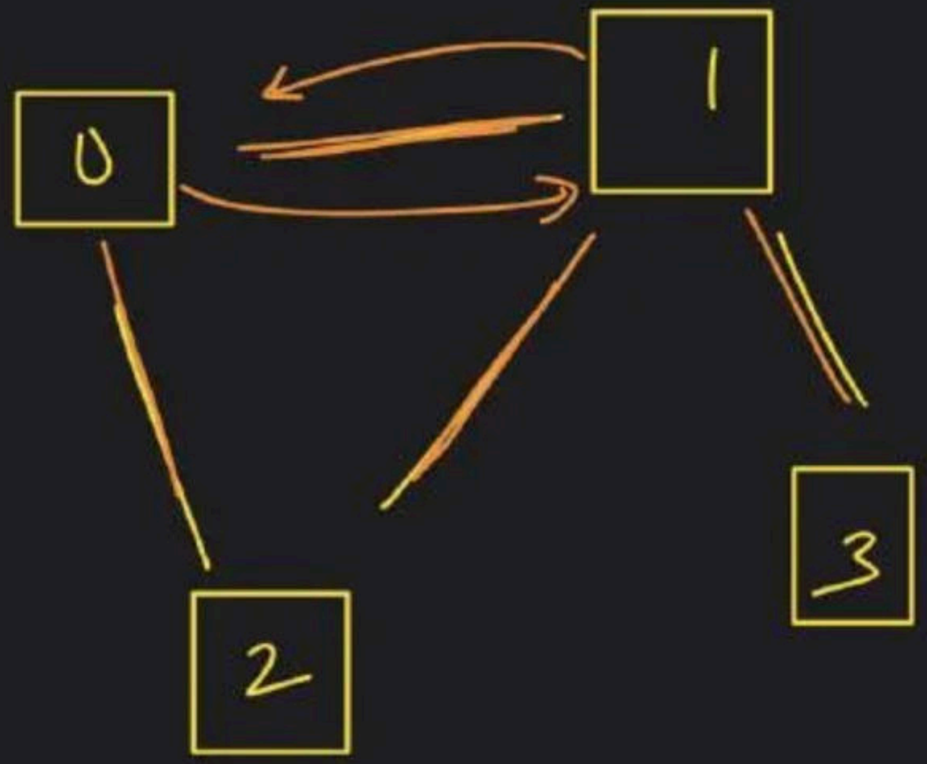


0 → ~~connected~~ not connected / edge

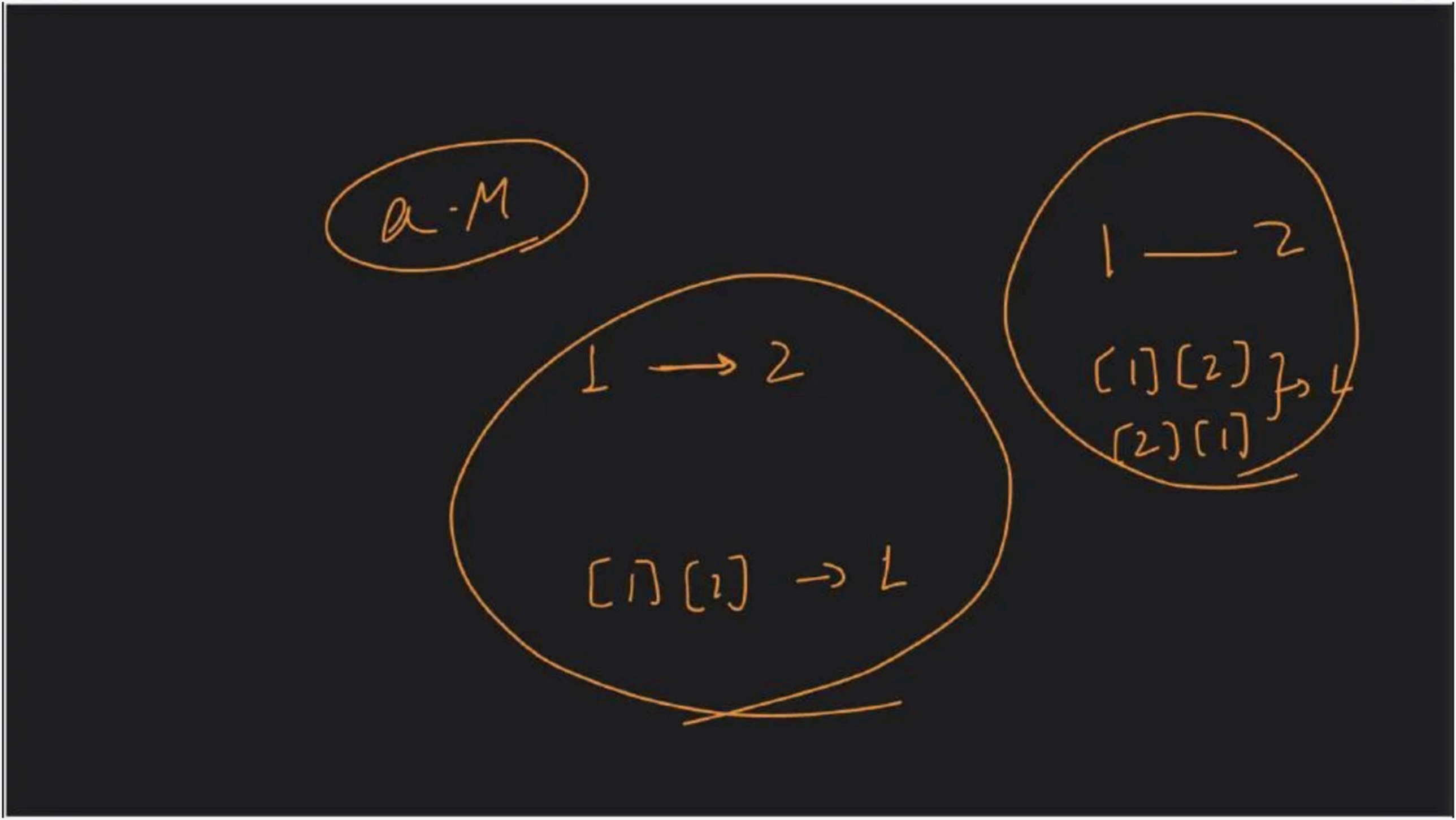
1 → ~~not connected~~ connected

0 — 1 [0][1] }1
[1][0]

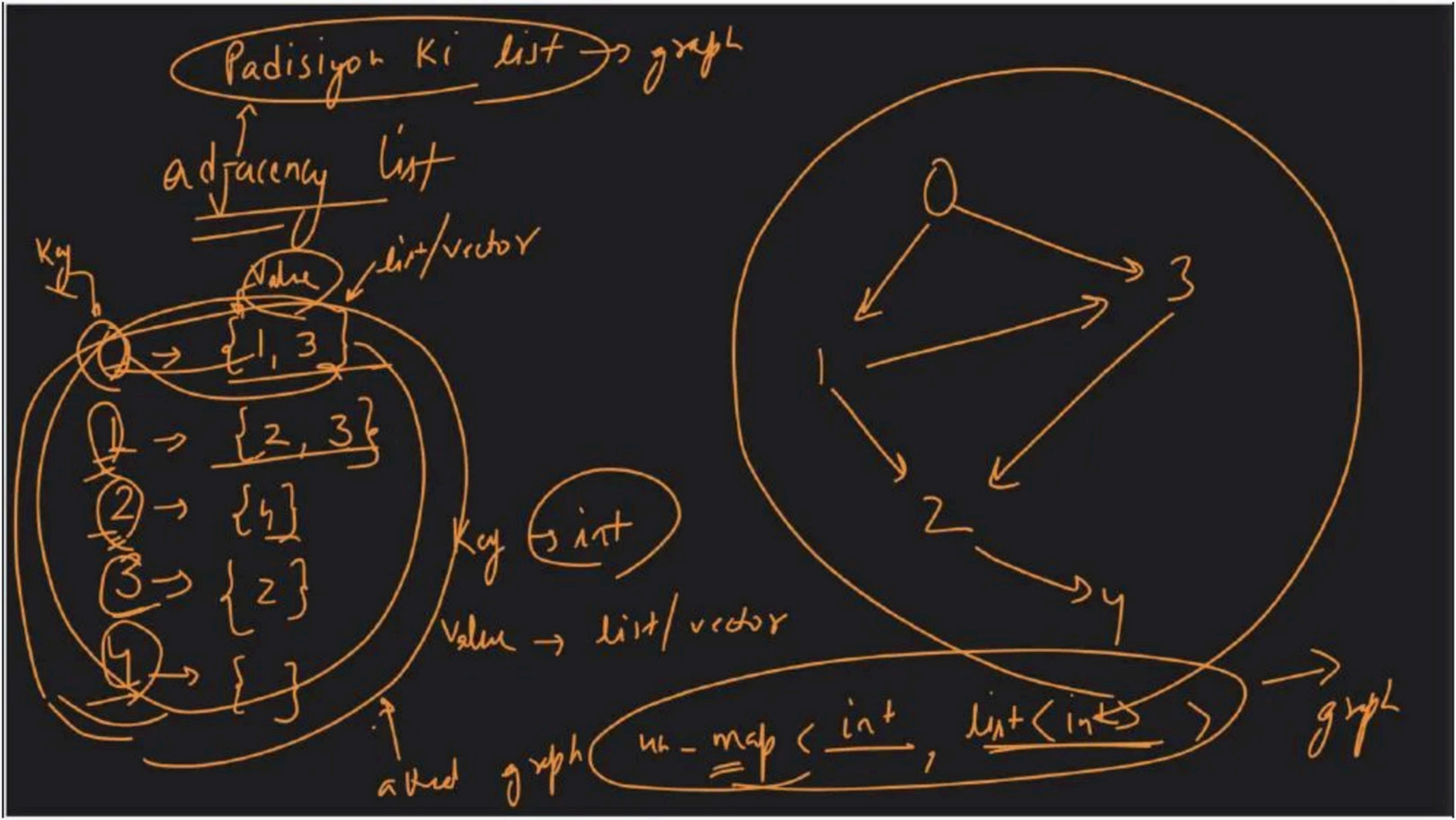




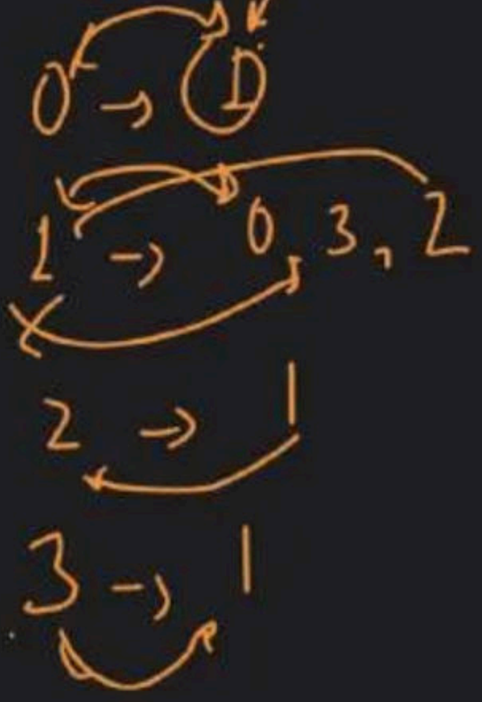
1-2 → [1][2]
[2][1]

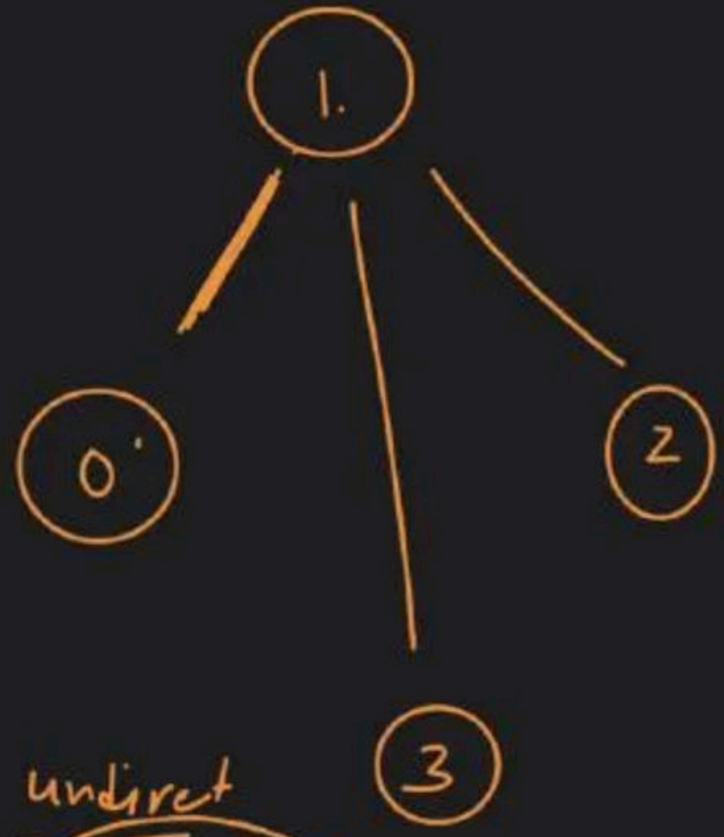

a·M
1 → 2
[1][1] → 1
1 — 2
[1][2] }
[2][1] }

→ Graph → Implementation → Adjacency List


Padisiyon Ki list → graph
adjacency list
Key
Value
list/vector
0 → {1, 3}
1 → {2, 3}
2 → {4}
3 → {2}
4 → { }
Key → int
Value → list/vector
adjacency graph
un-map < int , list<int> >
graph
0
1
2
3
4

Adj. list





directed

1 → 2

1 → {2}

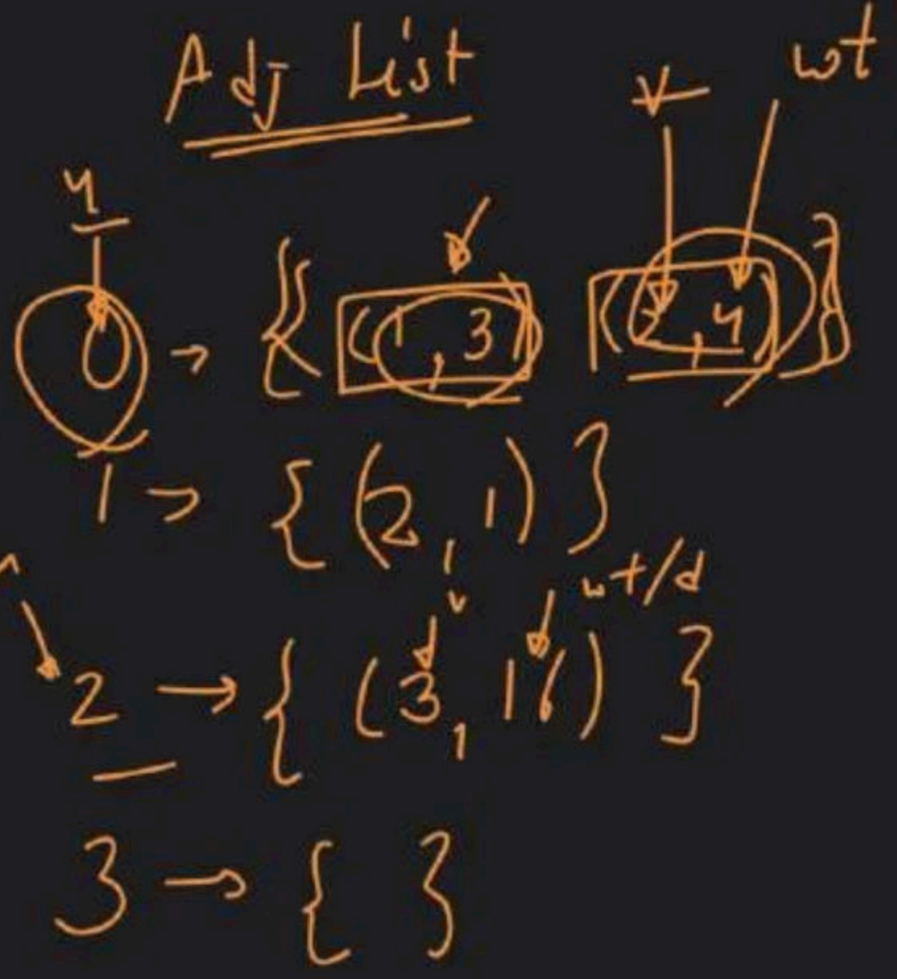
undirect

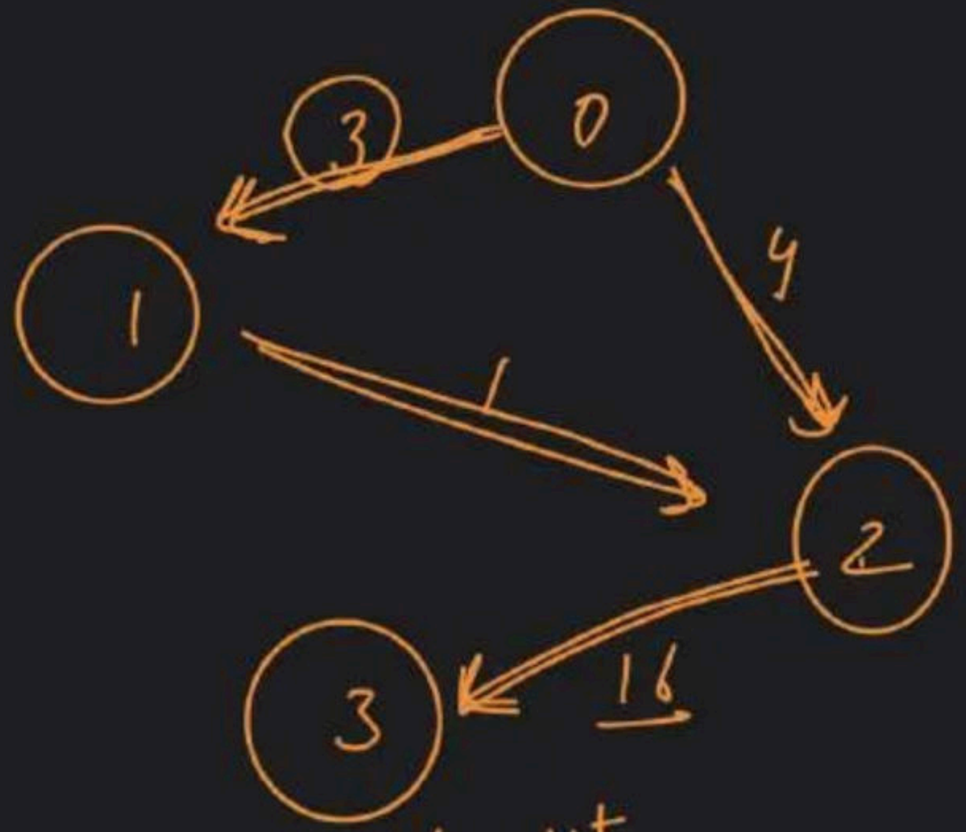
1 — 2

1 → {2}

2 → {1}

Adj List





unordered_map < int , list < pair < int , int > > > adjList

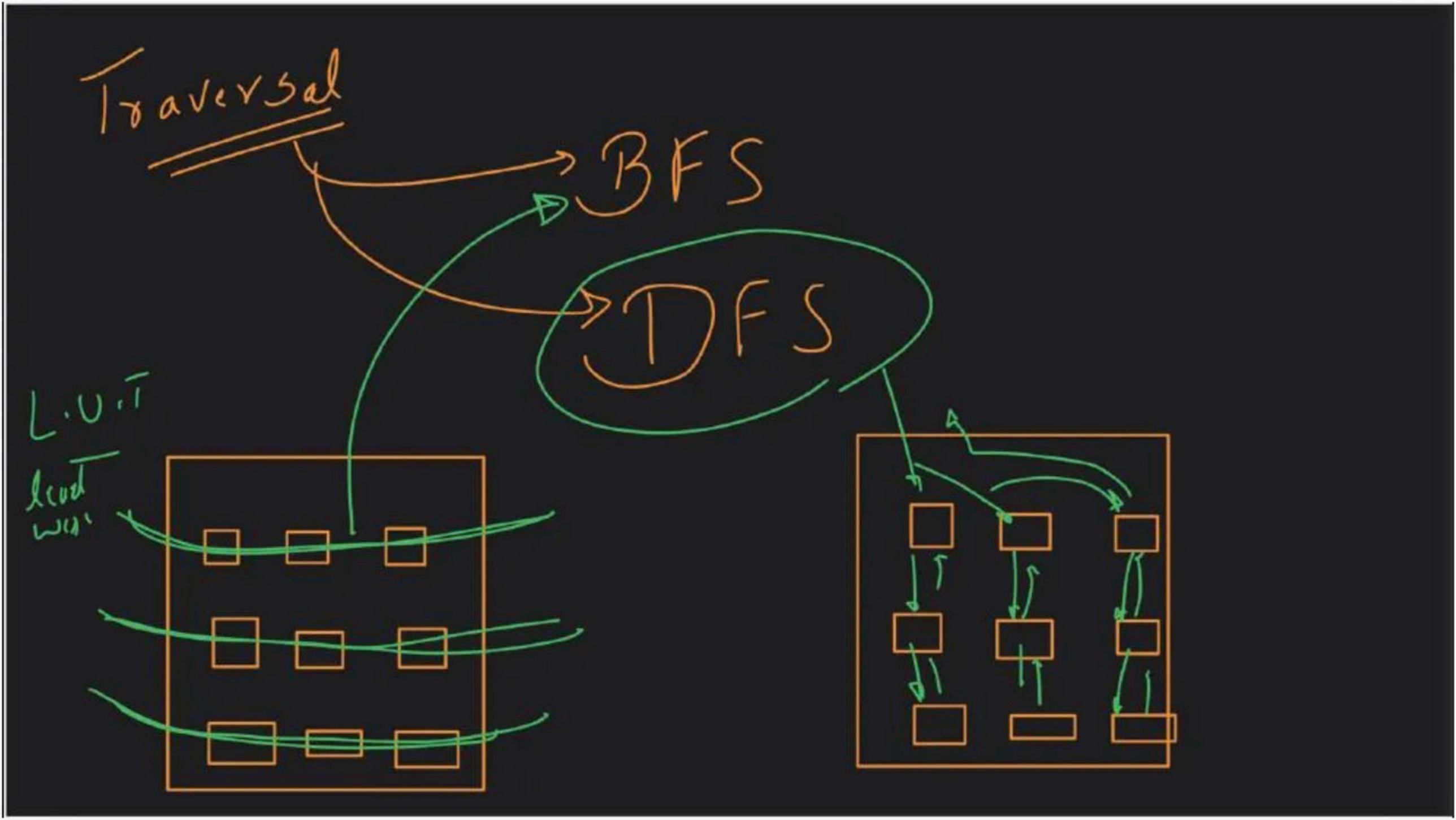
u v wt

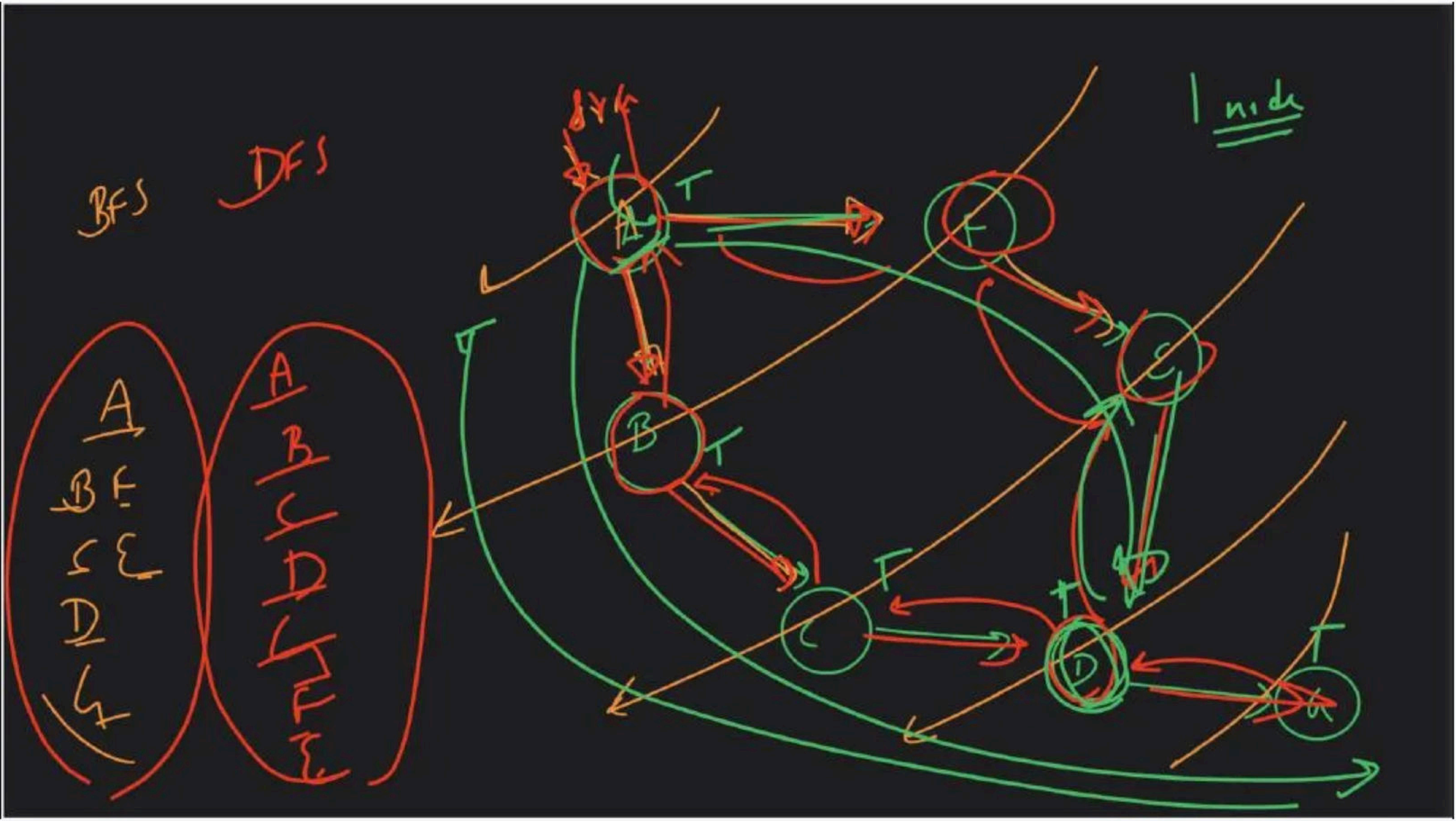
H/W

① Class Repeat → ②

② Code → T.C →

→ S.C →


Traversal
BFS
DFS
L.U.T


BFS
DFS
A
B F
C E
D
G
A
B
C
D
G
F
E

0 3 5 2 4 6 1

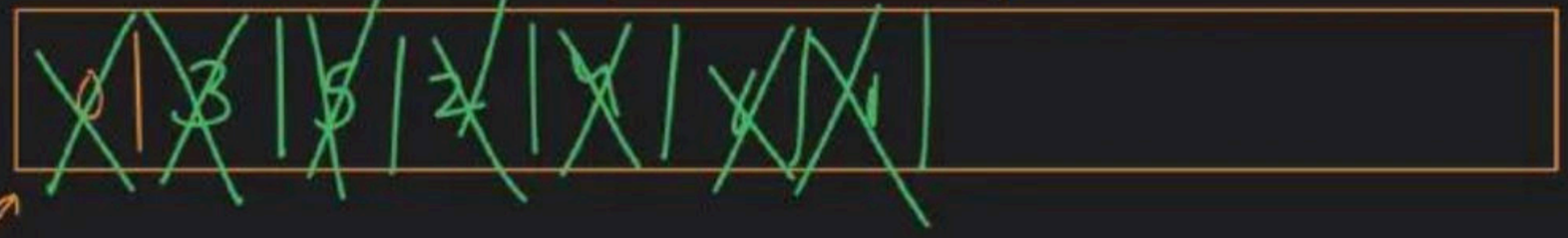
Queue

adj list

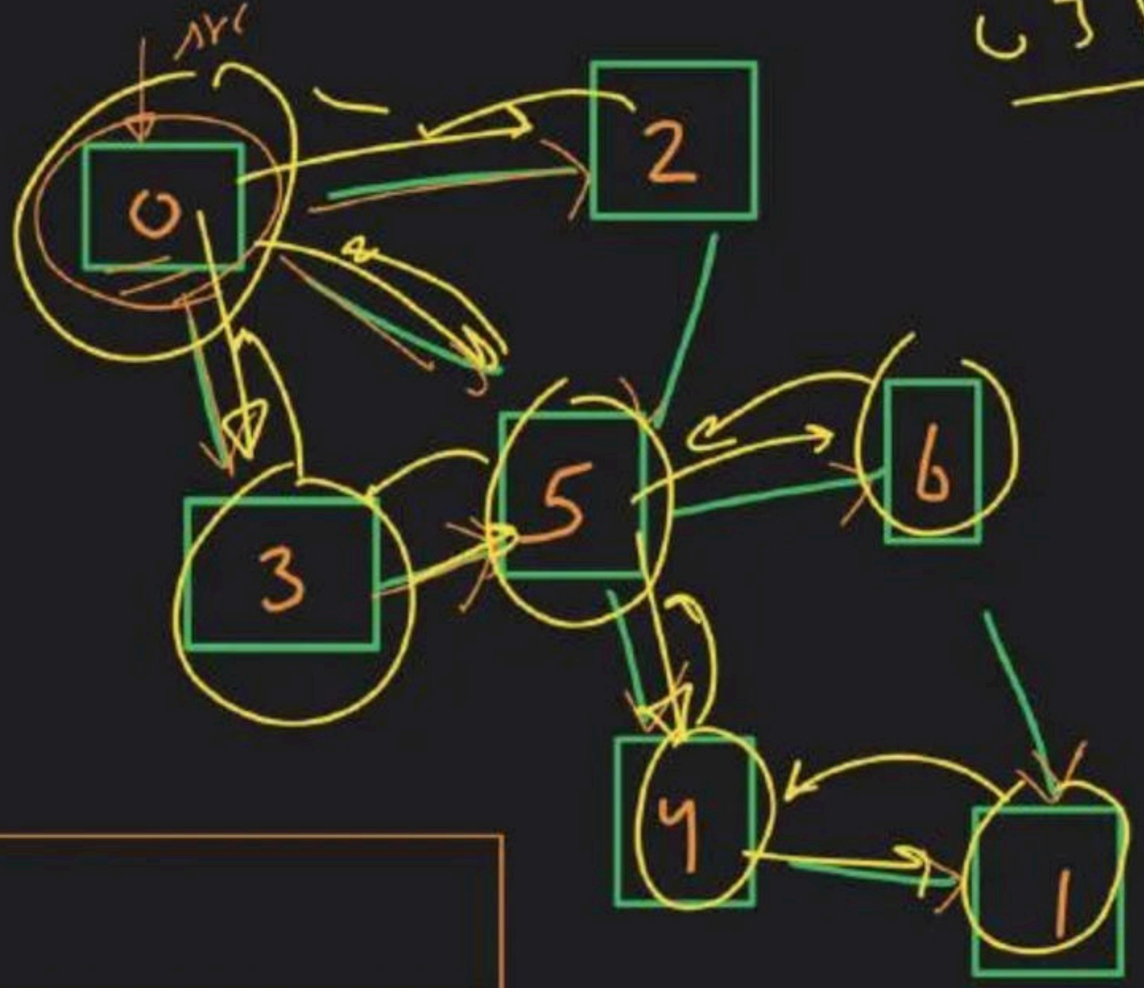
0 → {3, 5, 2}
1 → {1}
2 → {5}
3 → {5}
4 → {1}
5 → {4, 6}
6 → {1}

visited

0 → T
1 → T
2 → T
3 → T
4 → T
5 → T
6 → T



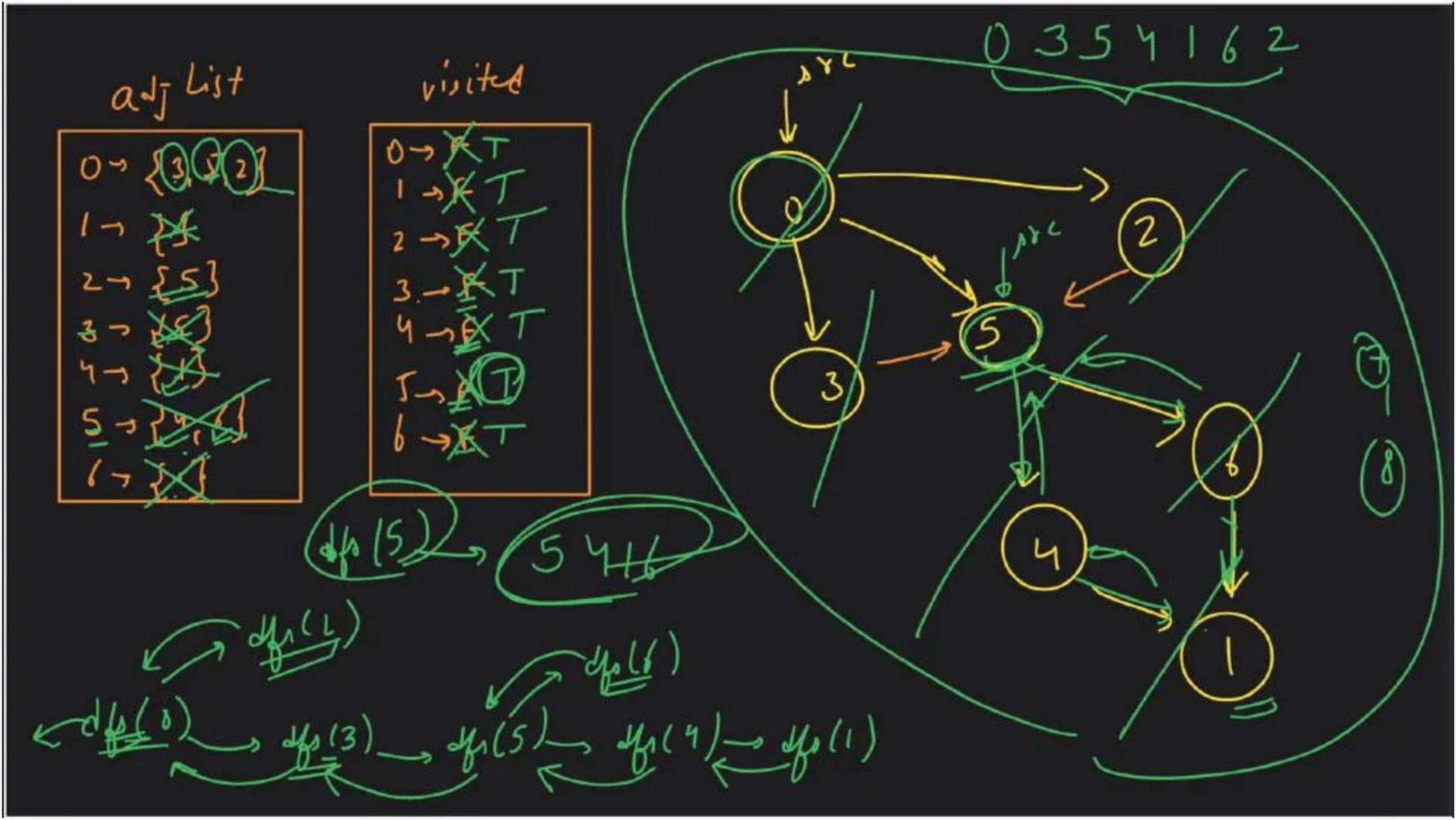
Q



0 3 5 4 6 2

① initial stuff → Q ← src, vis[src] = T

② main logic → front → neighbour


adj List
0 → {3, 5, 2}
1 →
2 → {5}
3 →
4 →
5 →
6 →
visited
0 → F T
1 → F T
2 → F T
3 → F T
4 → F T
5 → T
6 → F T
0 3 5 4 1 6 2
dfs(5)
5 4 1 6
dfs(1)
dfs(6)
dfs(0)
dfs(3)
dfs(5)
dfs(4)
dfs(1)

0 1 2

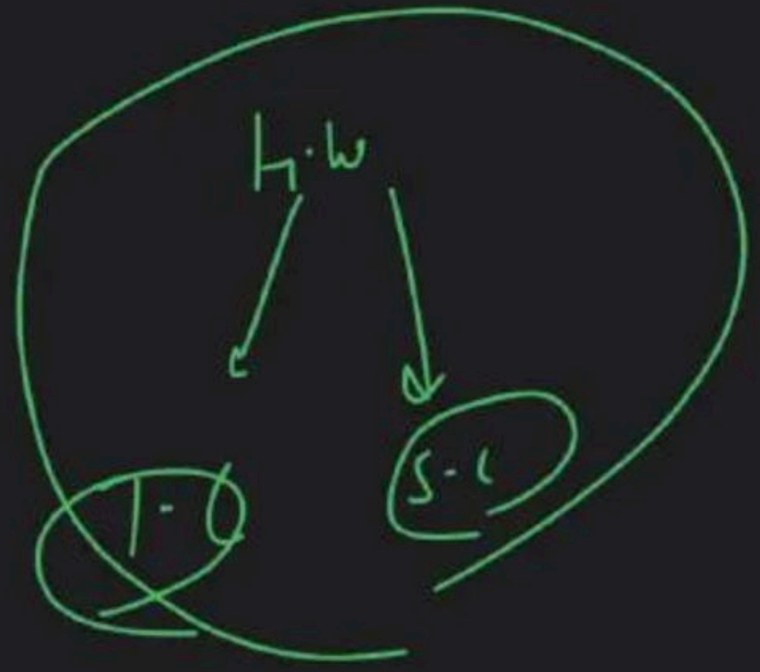
0 → {1}

1 → {0}

visited

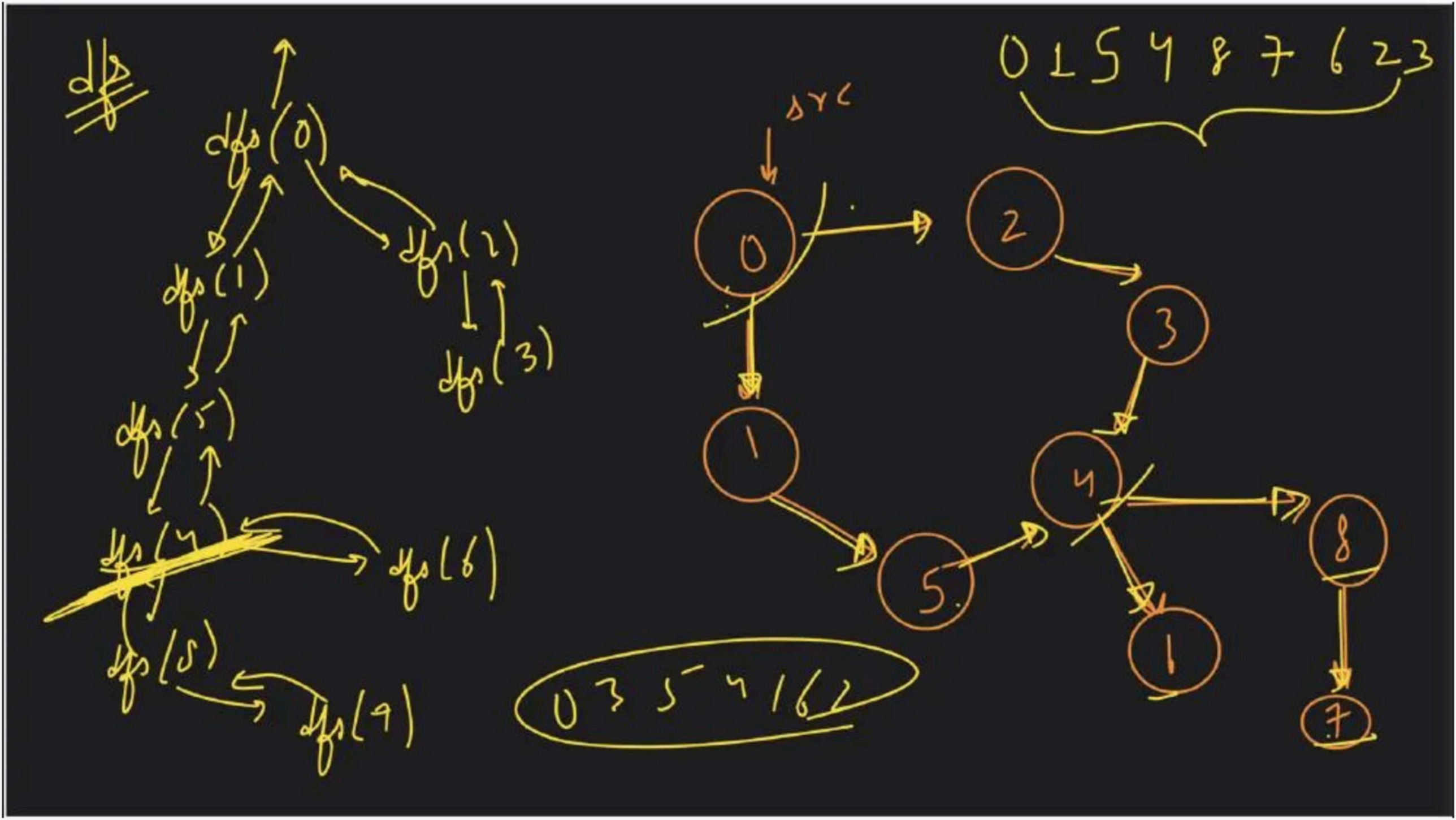
q ← ~~0~~ ~~1~~ ~~2~~ ~~3~~

~~4~~



DPs — extra

Kal
class
hai


dfs
dfs(0)
dfs(1)
dfs(2)
dfs(3)
dfs(4)
dfs(5)
dfs(6)
dfs(7)
dfs(8)
dfs(9)
src
0
1
2
3
4
5
6
7
8
0 1 5 4 8 7 6 2 3
0 3 5 4 6 1 6 2

Graph Class - 2

Special class

Love Babbar • Sept 26, 2024

no. of nodes → V

adj Matrix

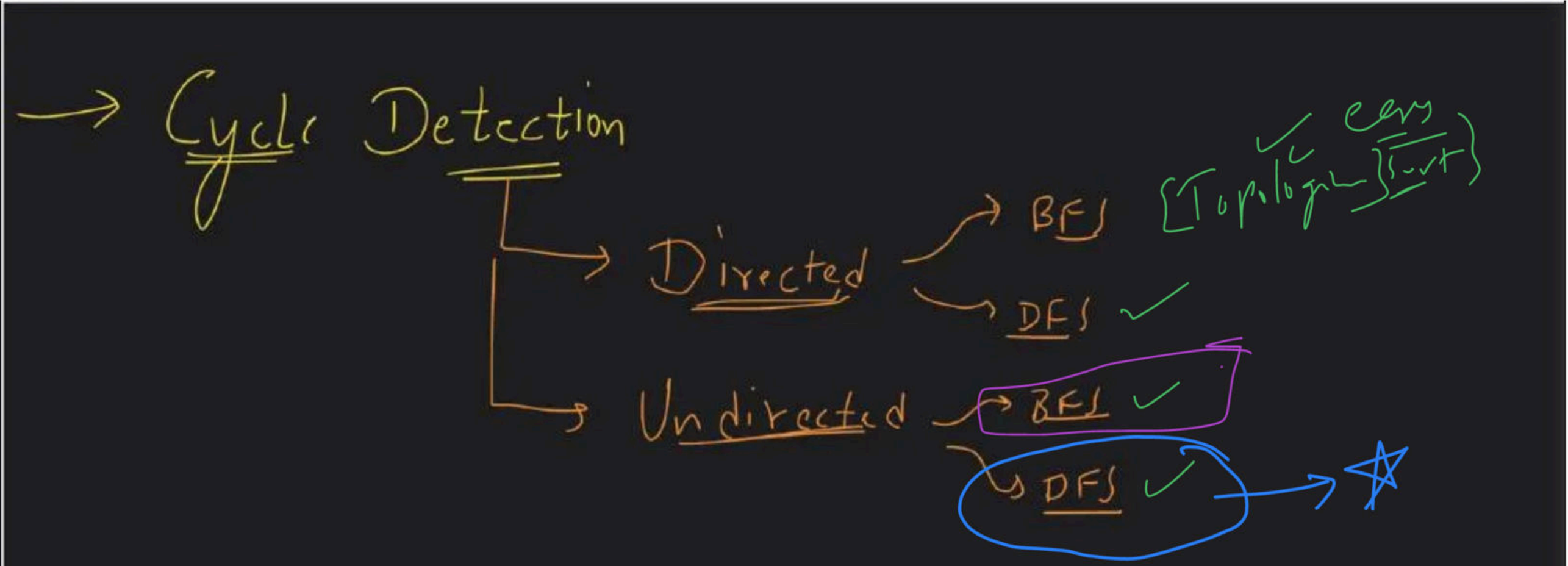
- T.C → $O(V^2)$
- S.C → $O(V^2)$

BFS & DFS

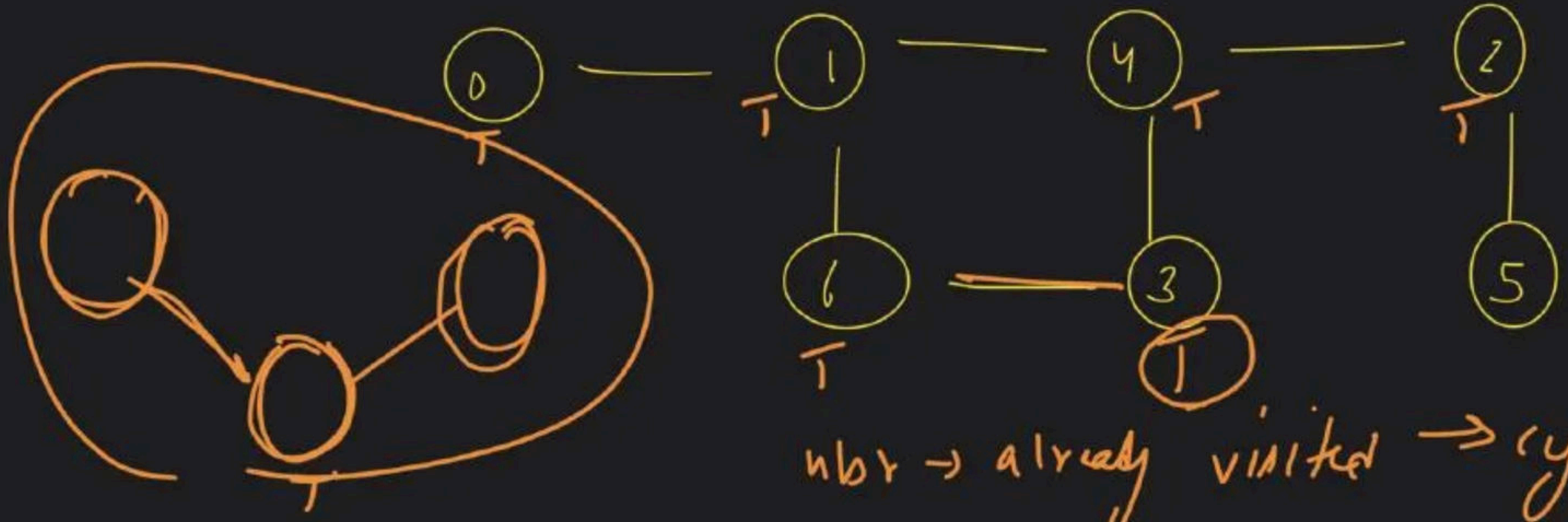
- T.C → $O(V+E)$
- S.C → $O(V)$

adj List

- T.C → $O(V+E)$
- S.C → $O(V+E)$

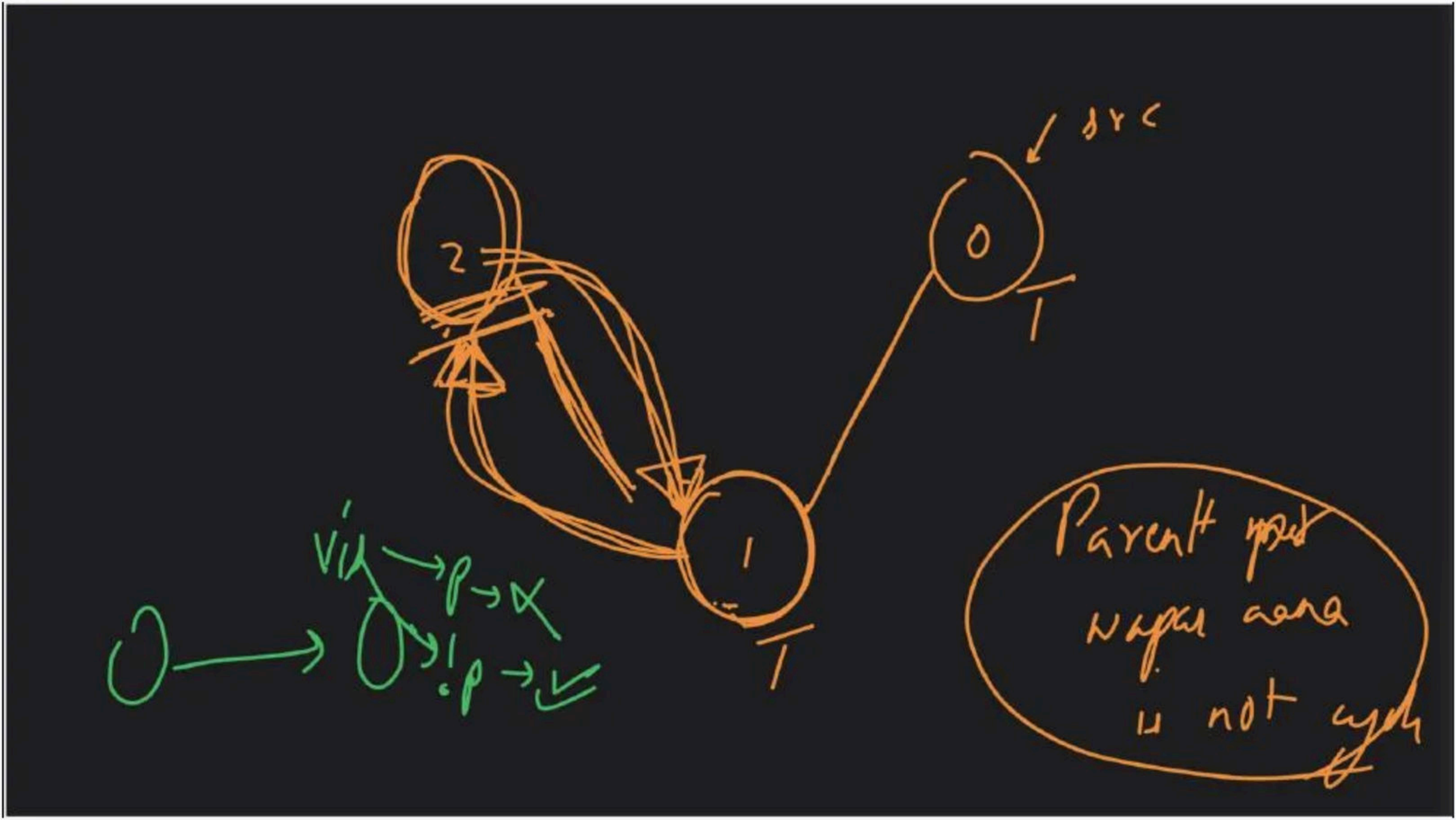

Cycle Detection
Directed
BFS
DFS
Topological sort
very
Undirected
BFS
DFS

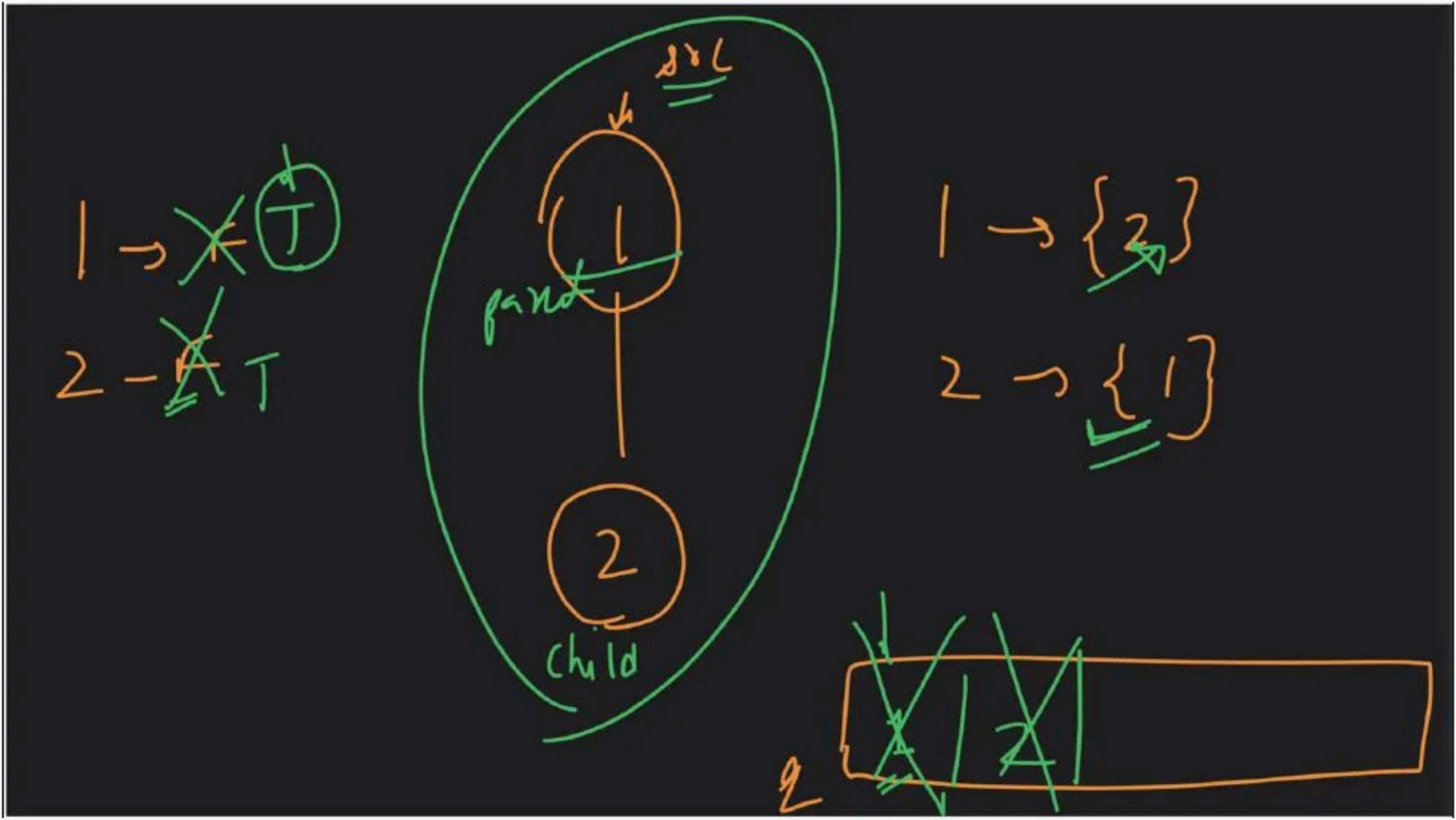
① Undirected → BFS

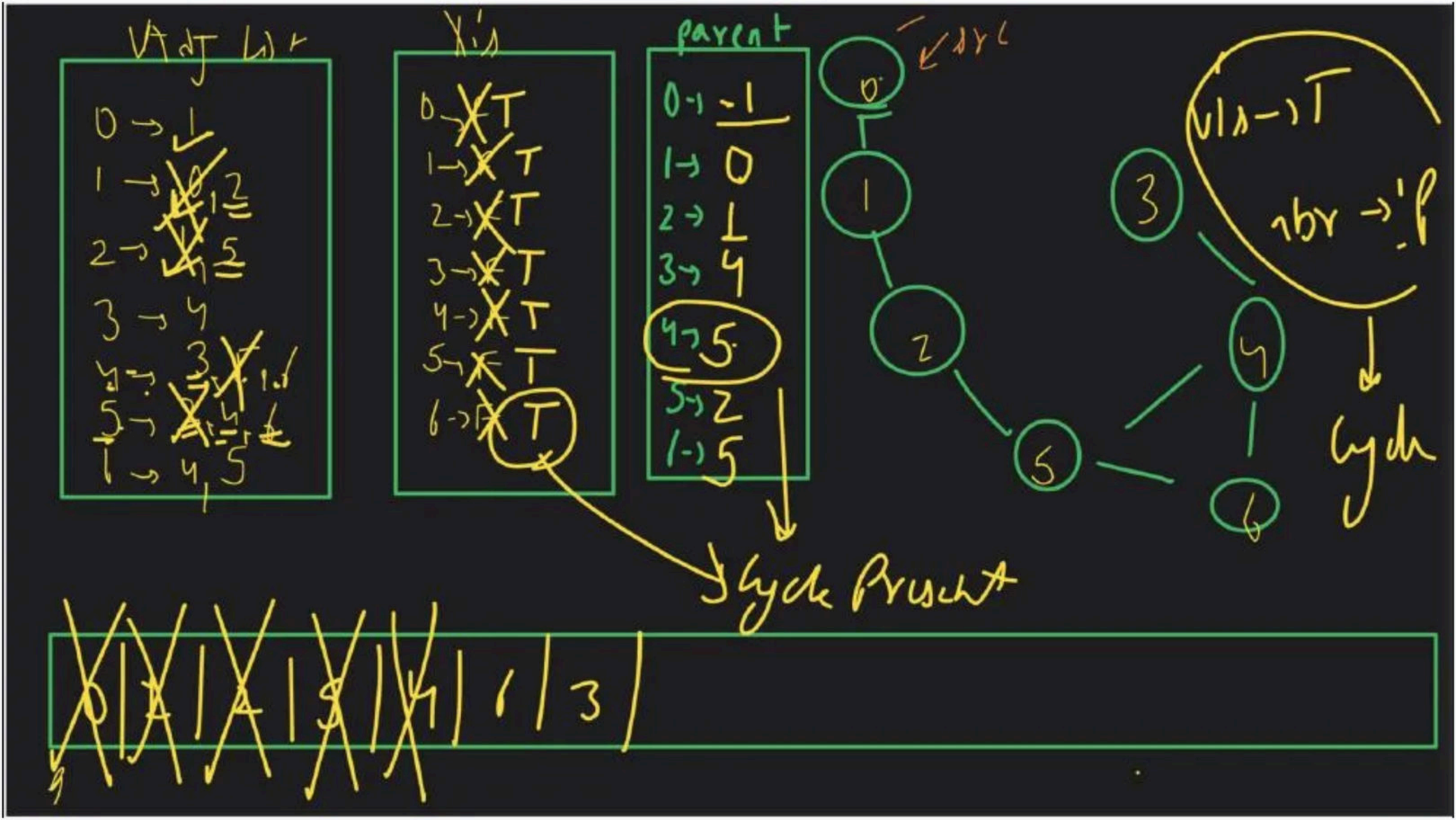


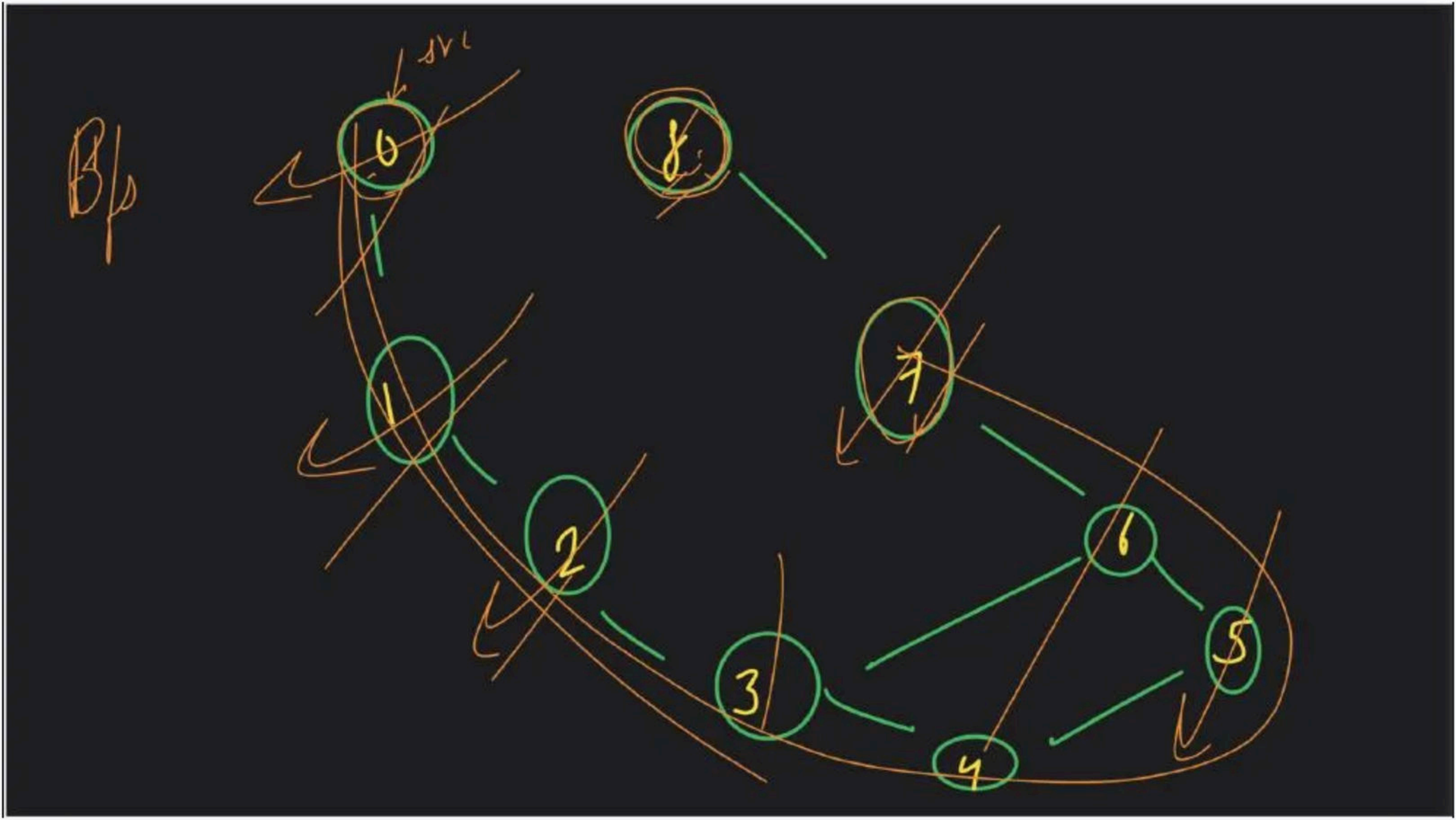
nbr → already visited → cycle

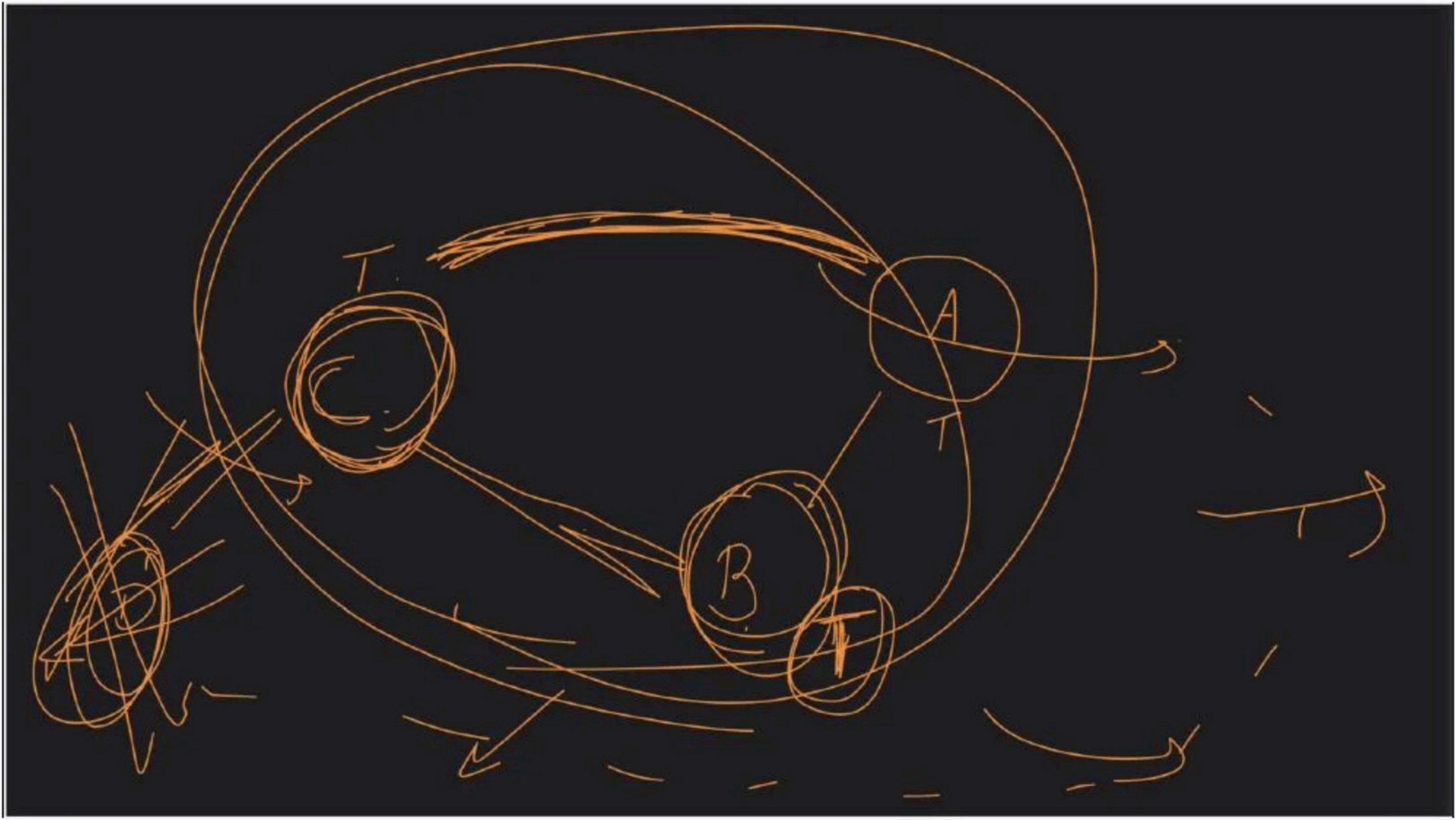
Break

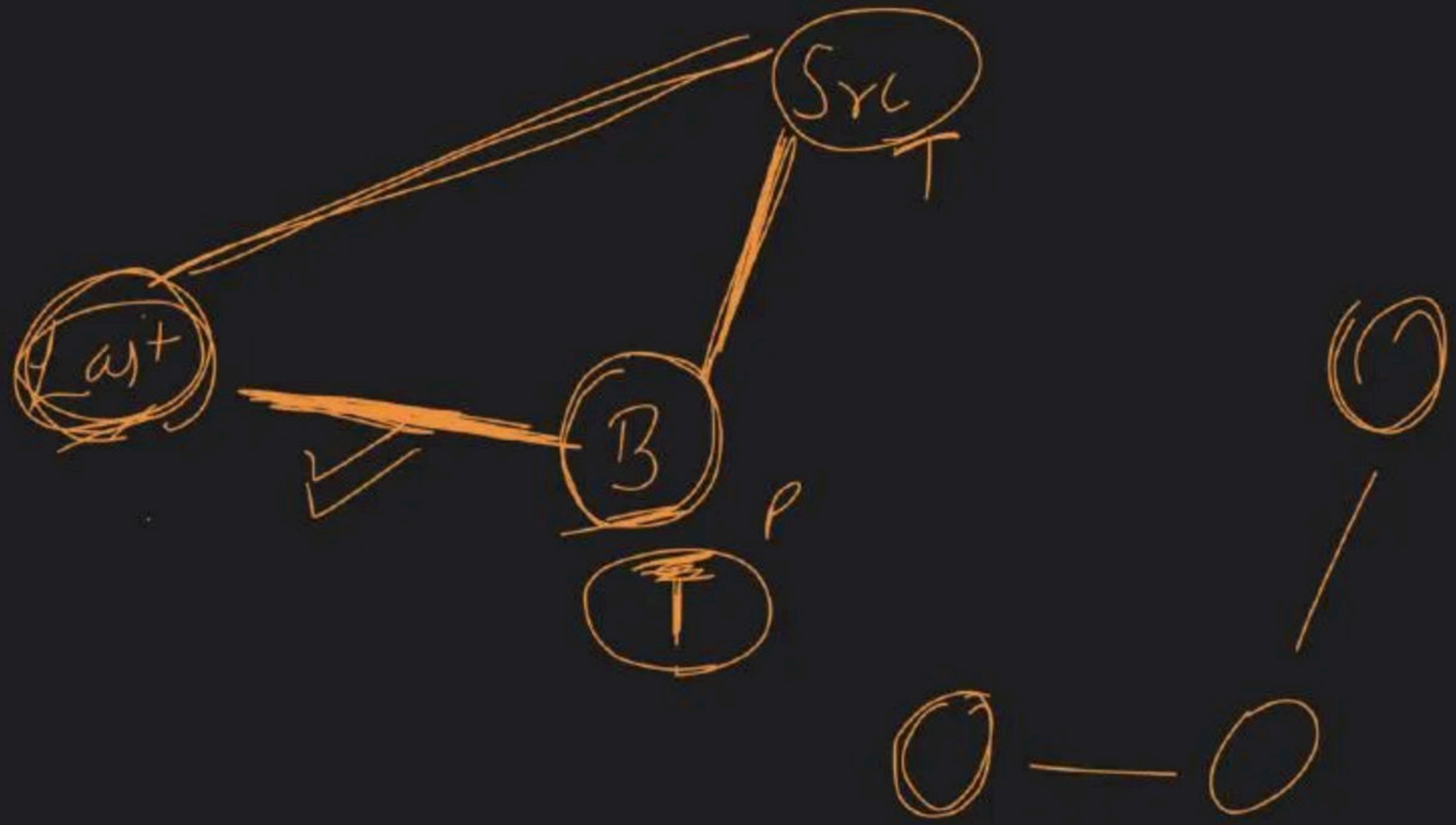

src
2
0
1
1
Parent port
not

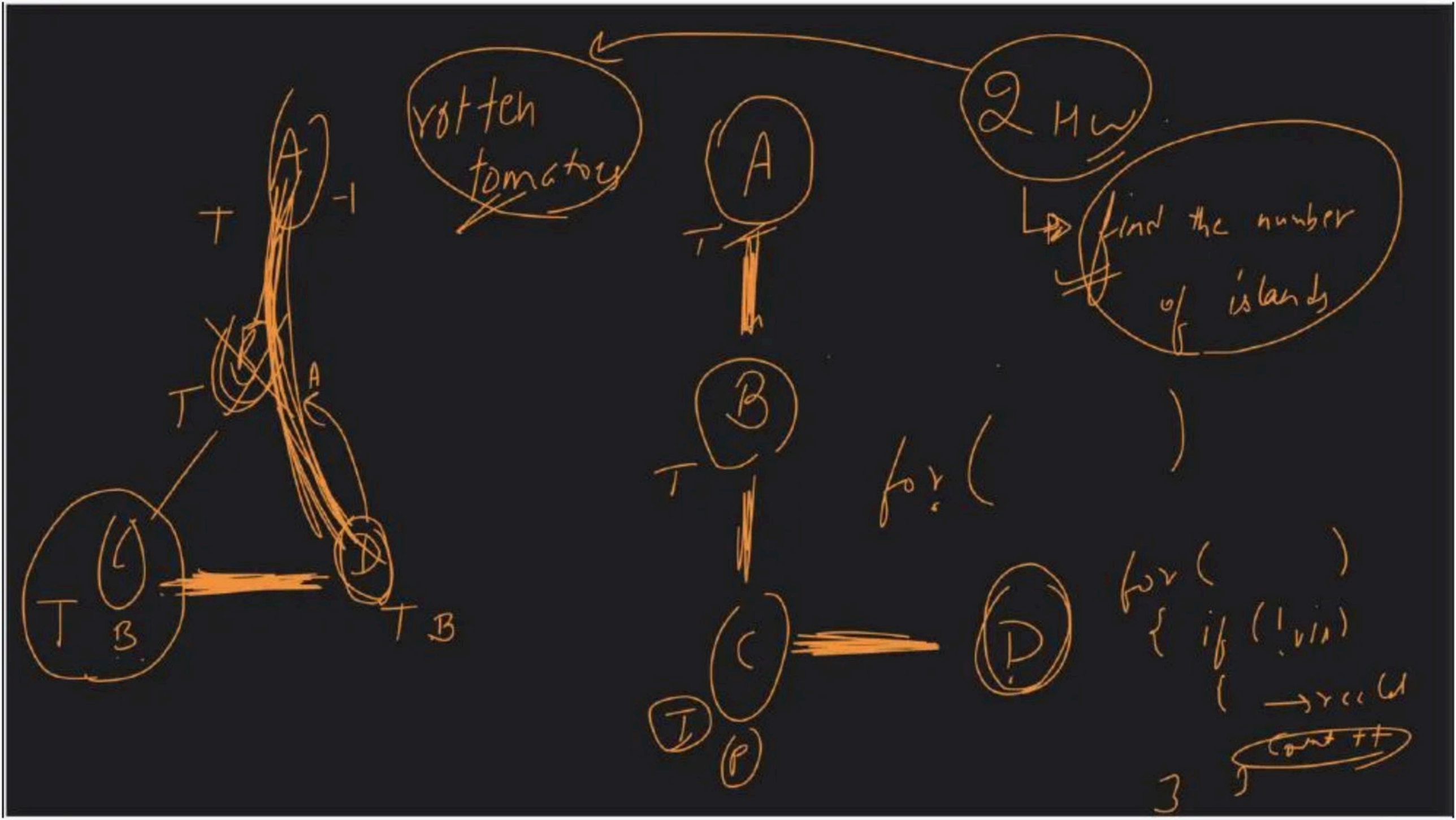

1 → T
2 → T
1
parent
2
child
1 → {2}
2 → {1}
2
1
2


parent
0 -1
1 0
2 1
3 4
4 5
5 2
1 5
Cycle Present
Cycle

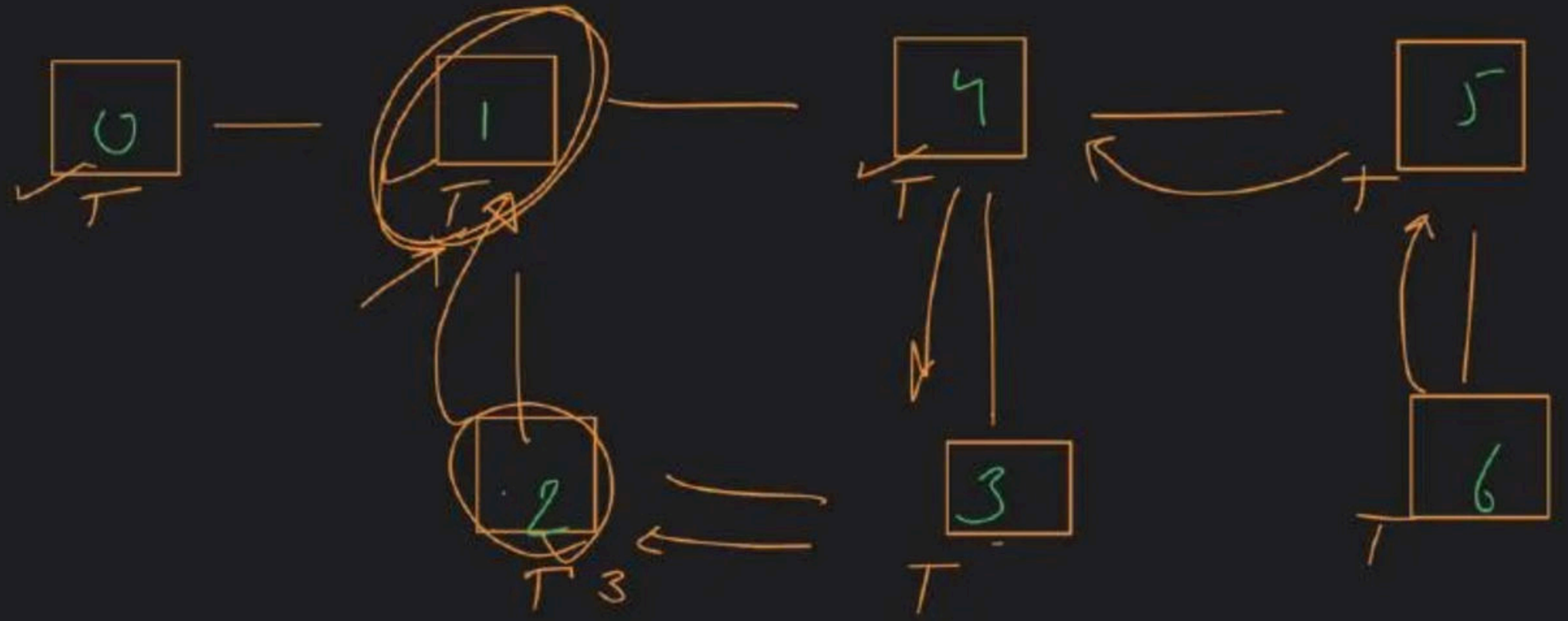

BFS
0
8
1
7
2
6
3
5
4

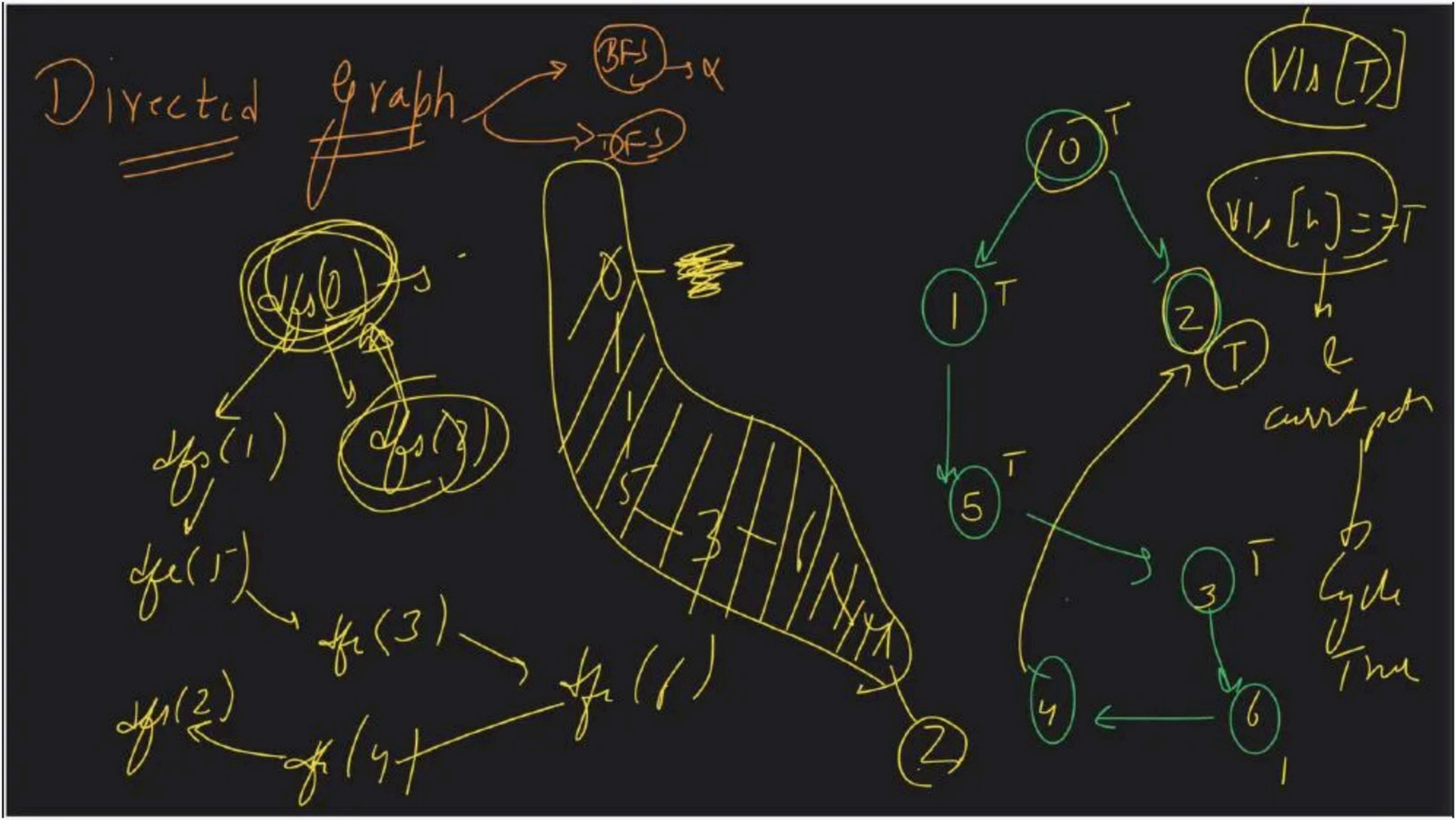

A
B

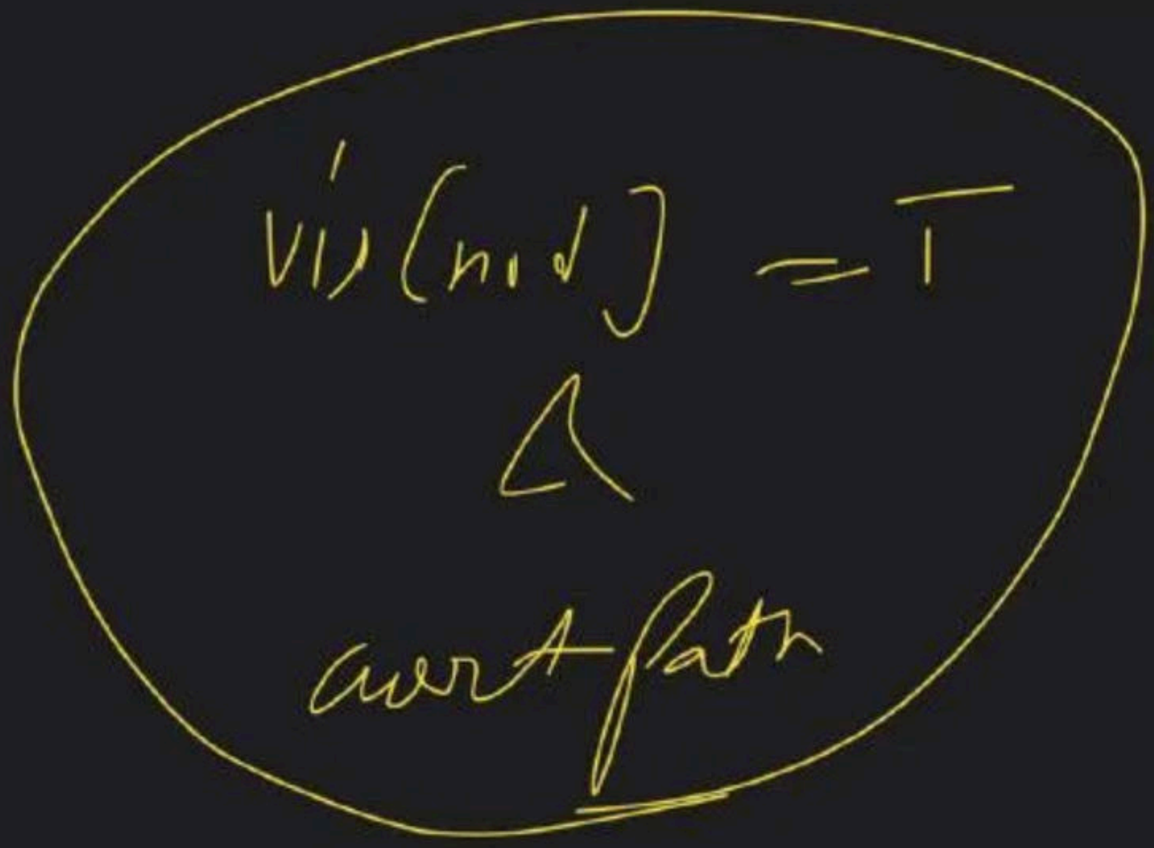

Src
T
alt
B
P
T


rotten tomatoes
2 HW
find the number of islands
A
B
C
D
T
B
I
P
-1
for ( )
for ( )
{ if (
(
count ++
3

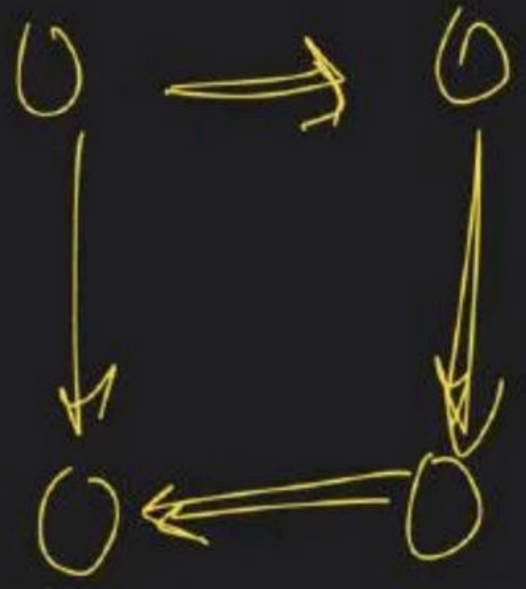
Undirected graph → DFS

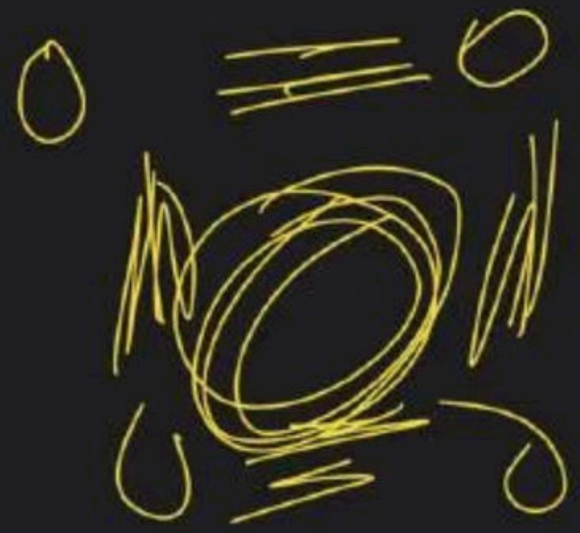


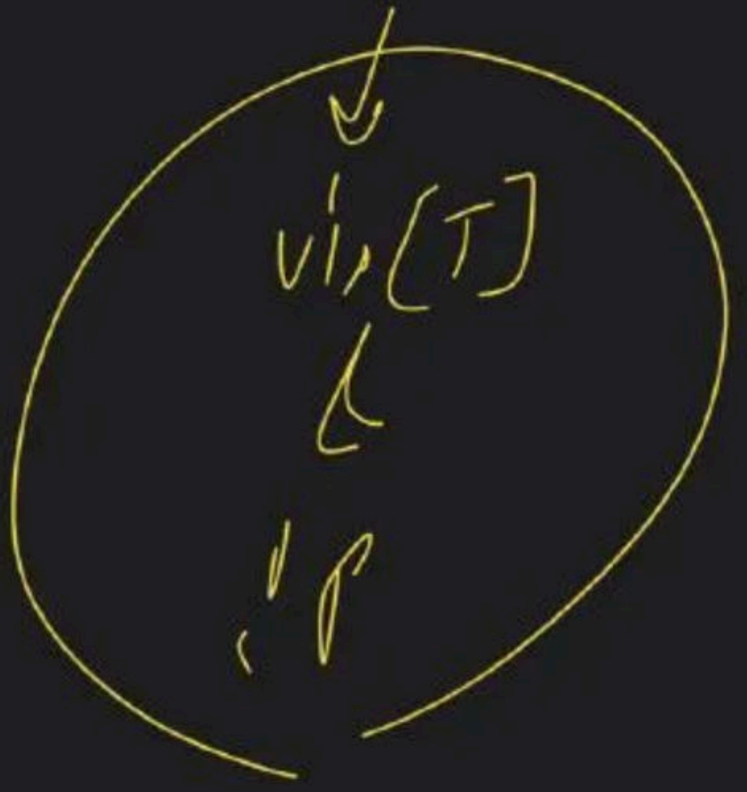

Directed Graph
BFS
DFS
dfs(0)
dfs(1)
dfs(3)
dfs(2)
dfs(4)
dfs(1)
dfs(3)
dfs(1)
dfs(4)
dfs(2)
0
1
5
3
4
2
VIS [T]
VIS [4] == T
current node
cycle
True
0 T
1 T
2 T
5 T
3 T
4
6
1

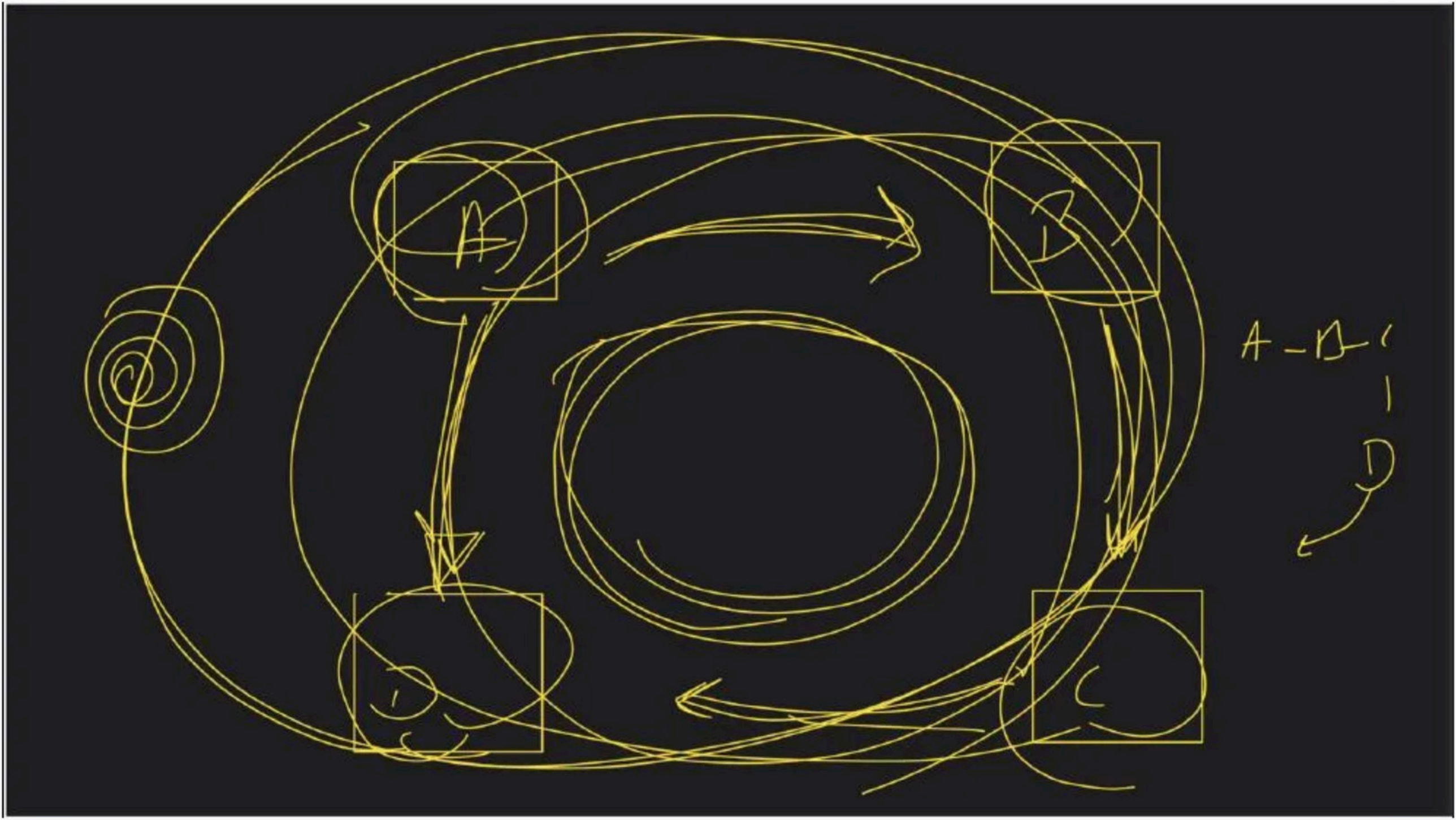

Vis[mid] = T
Δ
avatar

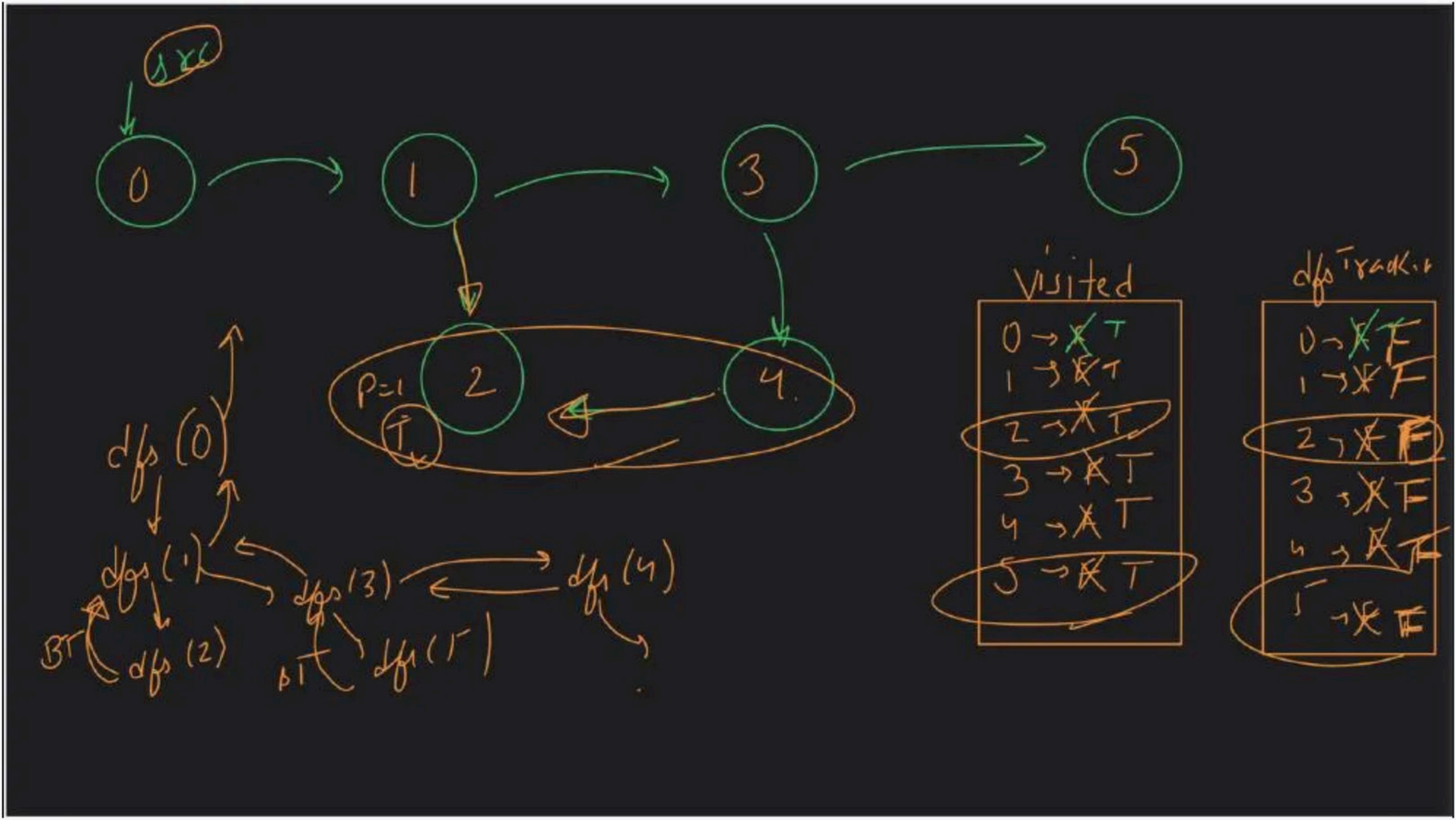
closed loop

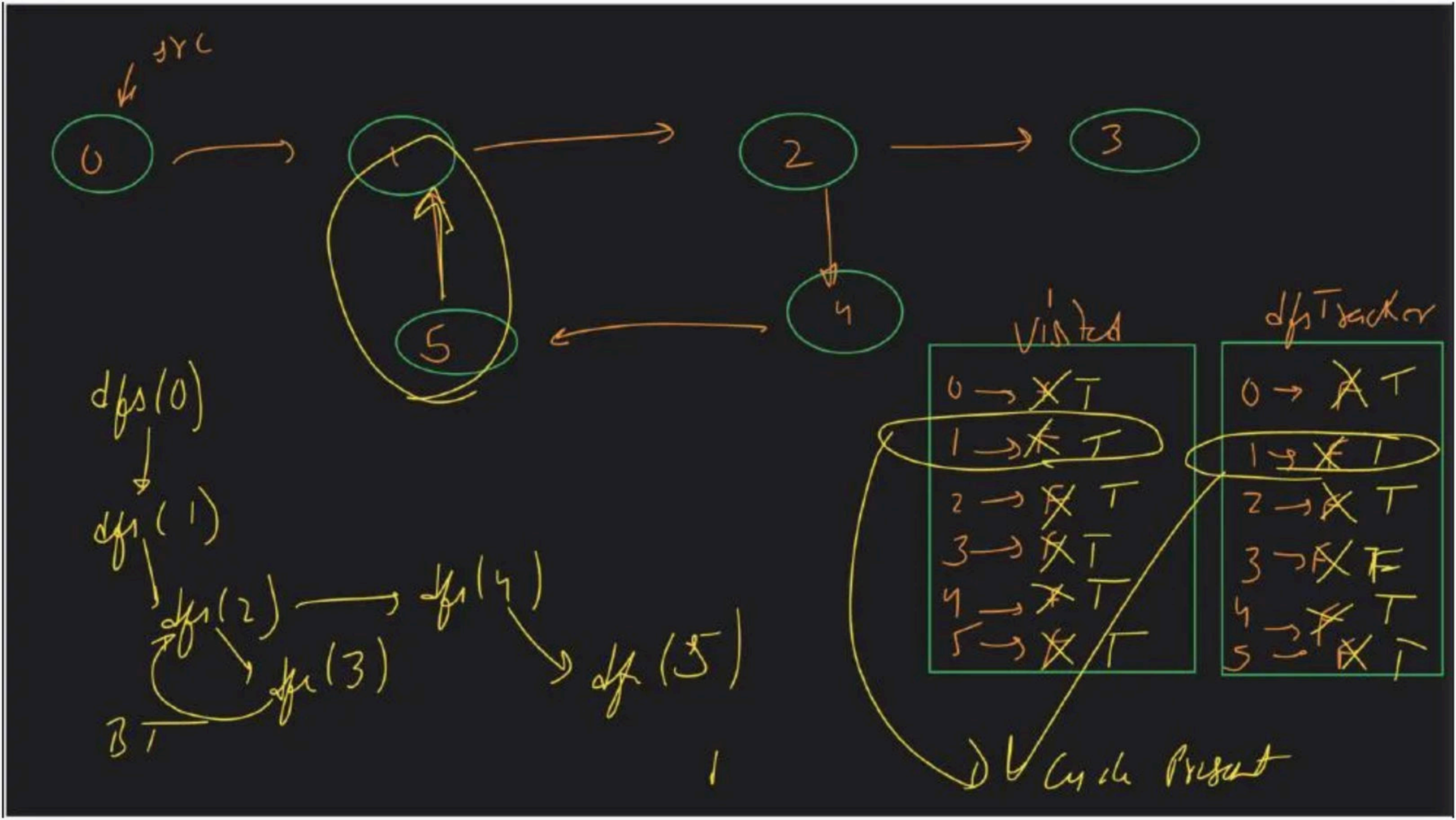






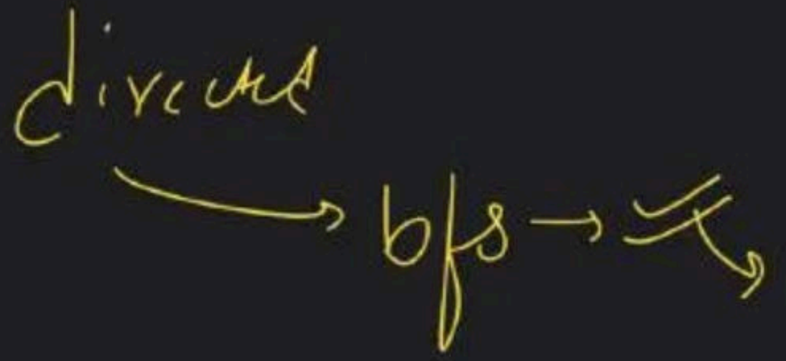

A
B
C
D
A-B-C-D


src
0
1
3
5
P=1
2
4
dfs(0)
dfs(1)
dfs(3)
dfs(4)
dfs(2)
dfs(1)
BT
BT
Visited
dfsTrack


src
0
1
2
3
4
5
dfs(0)
dfs(1)
dfs(2)
dfs(3)
dfs(4)
dfs(5)
BT
visited
dfsTracker
Cycle Present

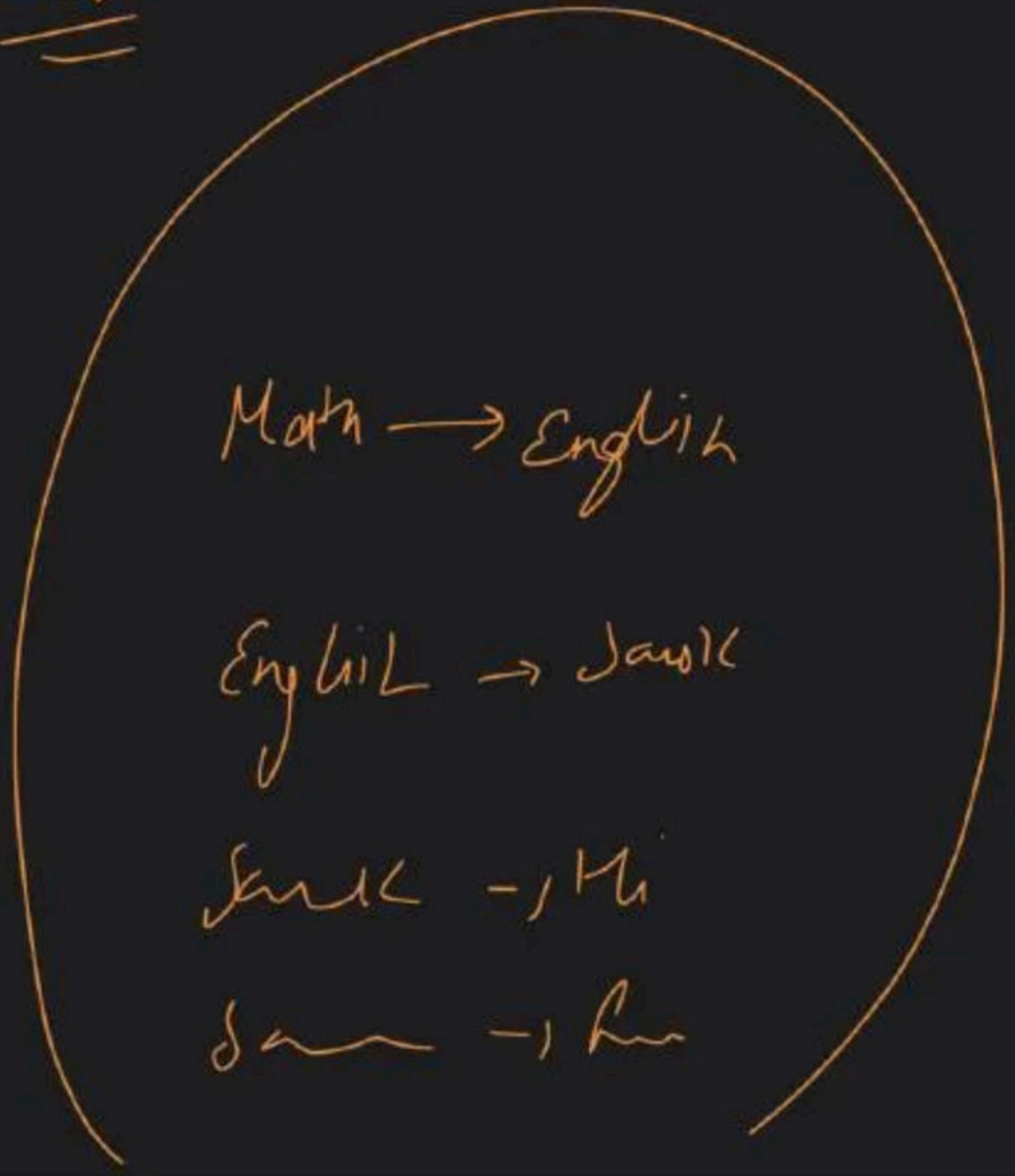
Graph Class - 3

Special class



Love Babbar • Sept 30, 2024

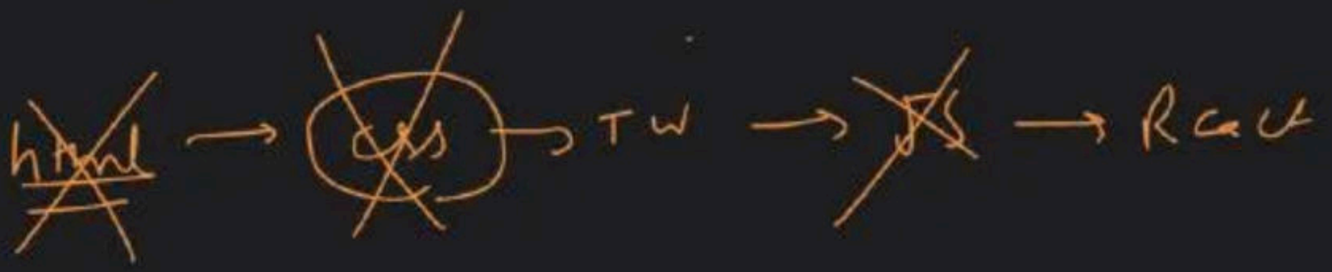
→ Topological Sort



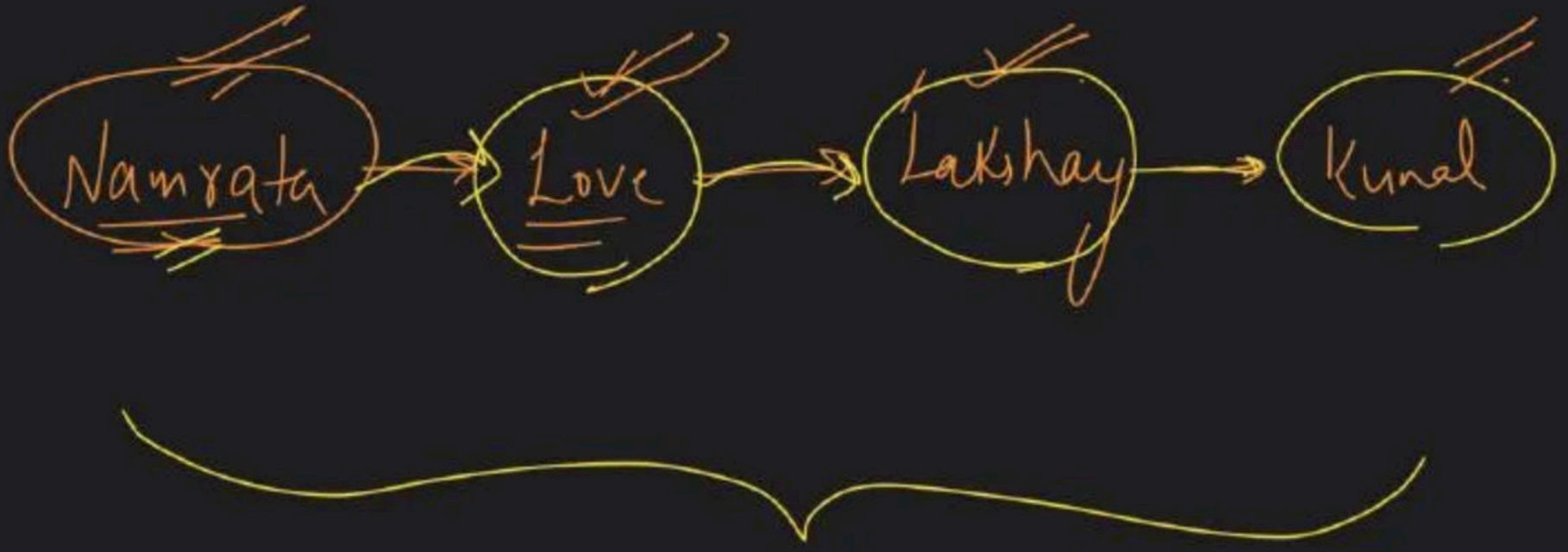
Flow

Hindi → P → J → C → M

web-dev



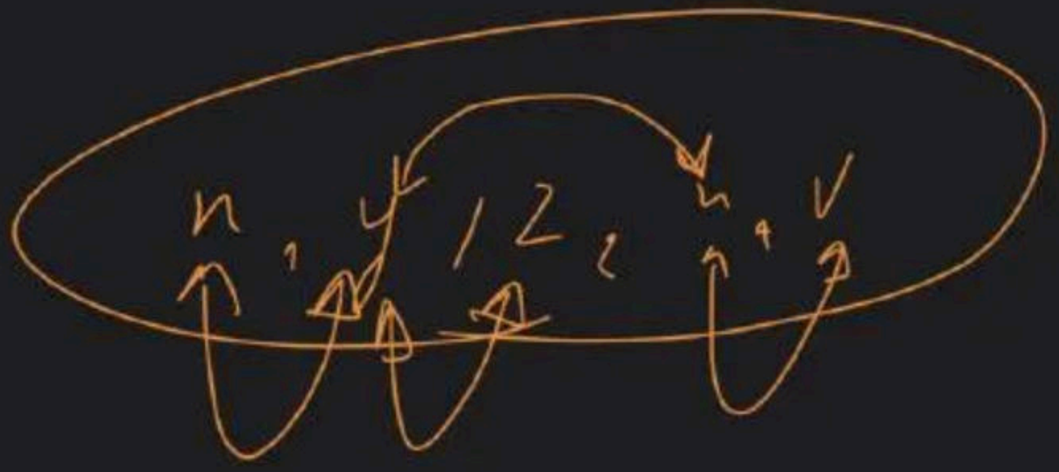
Notes



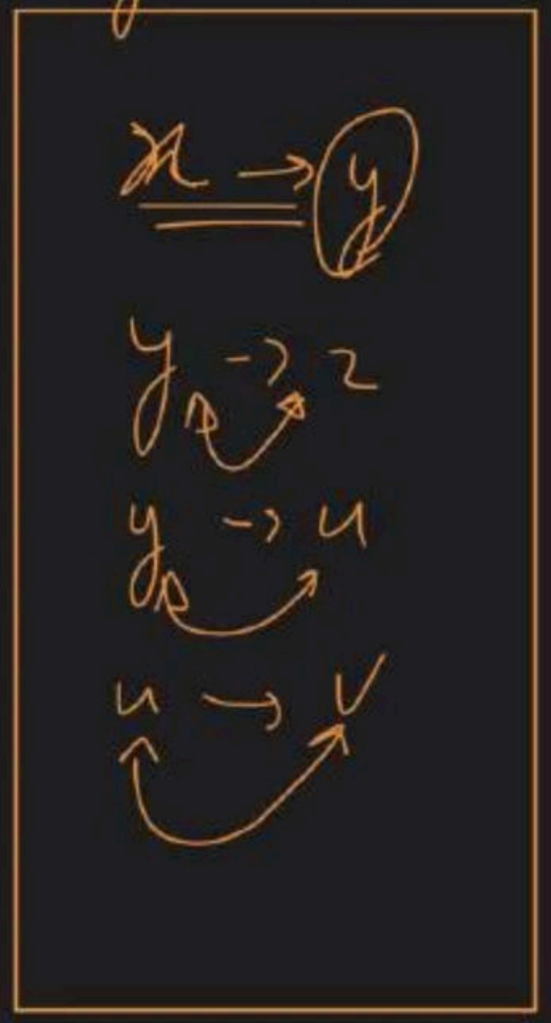
→ Topological Sort { DAG }

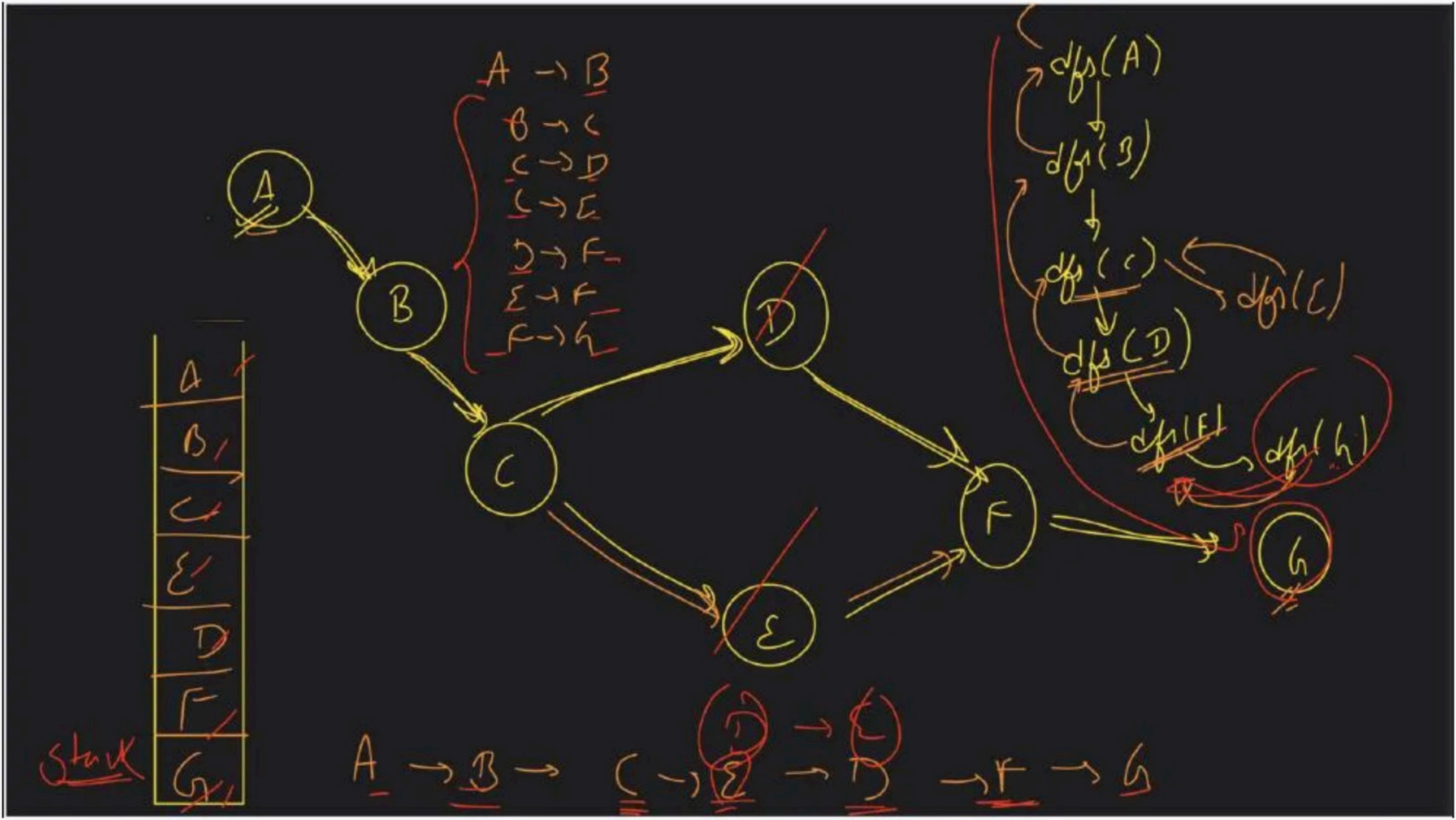
directed, Acyclic

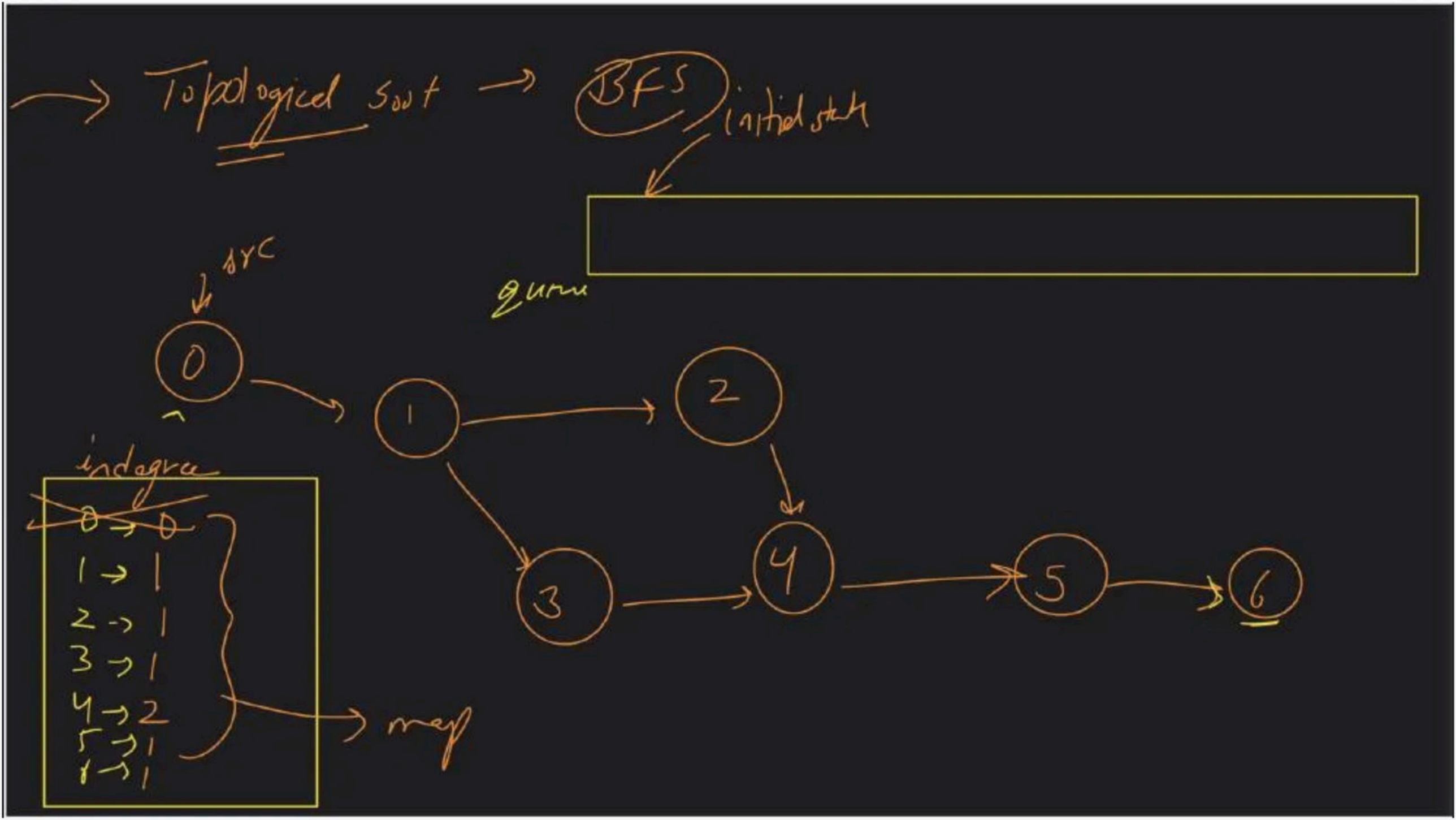
↳ Linear Ordering + rule



Edge List

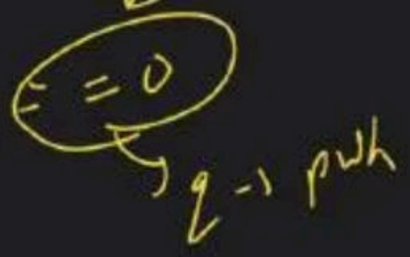



A → B
B → C
C → D
C → E
D → F
E → F
F → G
A
B
C
D
E
F
G
dfs(A)
dfs(B)
dfs(C)
dfs(D)
dfs(E)
dfs(F)
dfs(G)
dfs(E)
A
B
C
D
E
F
G
stack
A → B → C → E → B → F → G


Topological sort → BFS initial stack
src
queue
0
1
2
3
4
5
6
indegree
0 → 0
1 → 1
2 → 1
3 → 1
4 → 2
5 → 1
6 → 1
map

(1) indegree calculate

(2) travel indegree



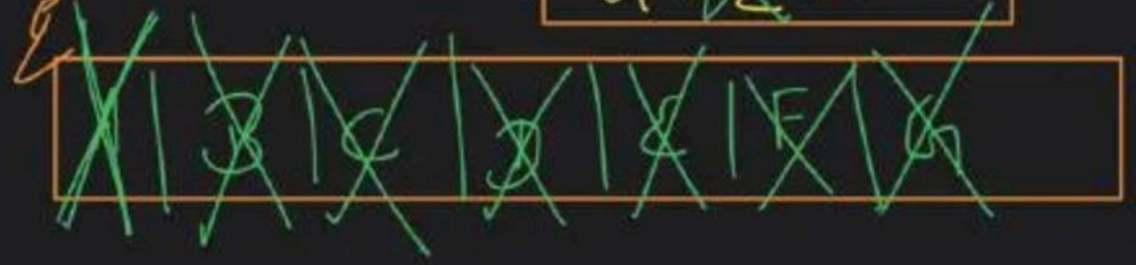
(3) main BFS logic

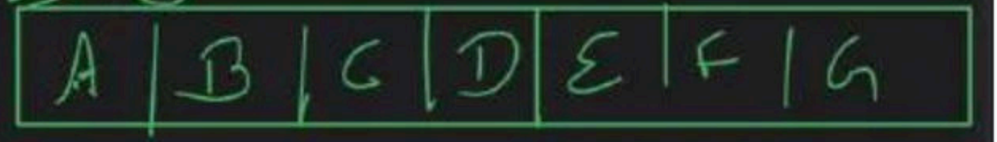
disconnected component

7

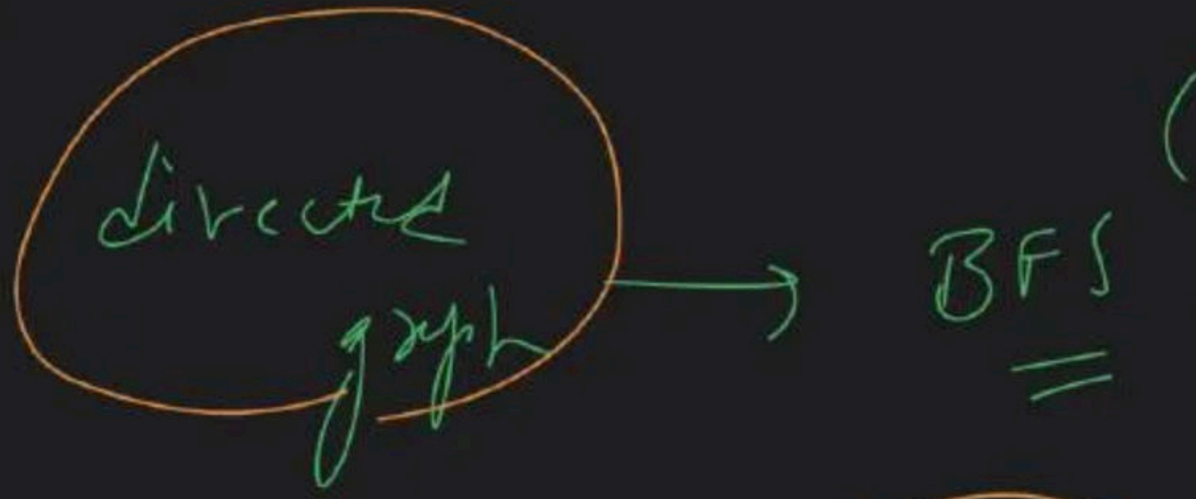
indegree

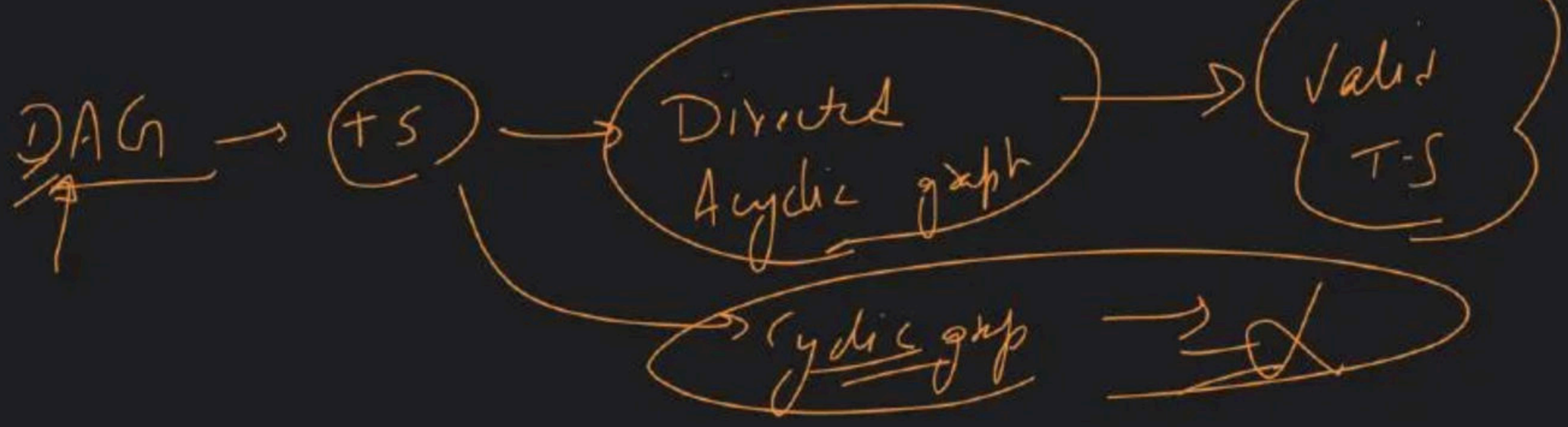
A → 0
B → 0
C → 0
D → 2 X 0
E → 0
F → 0
G → 2 0

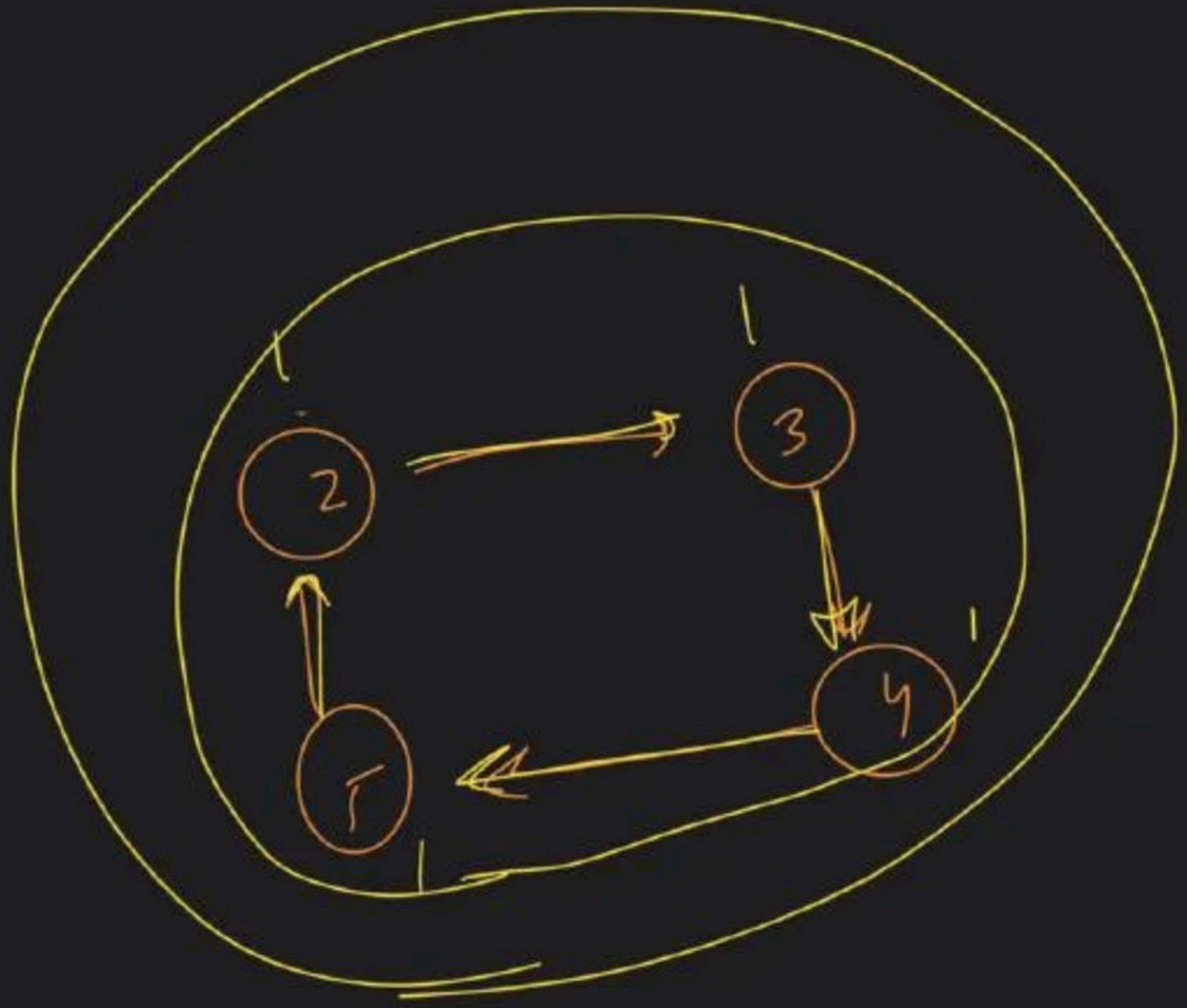


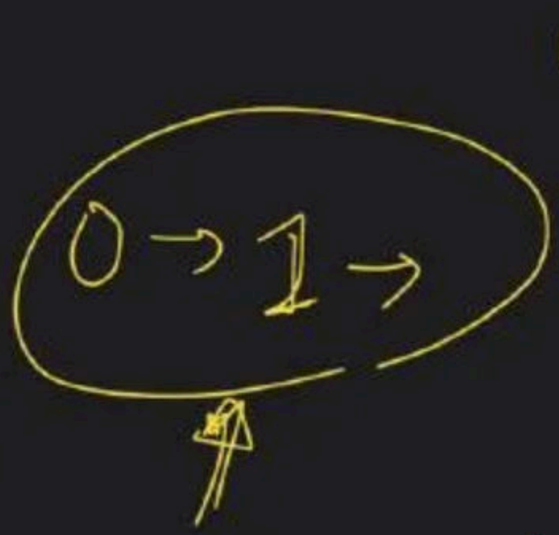


Cycle detection

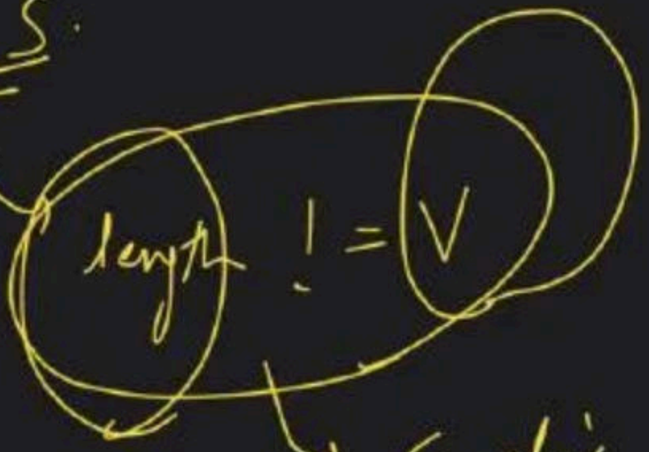








T.S.



cyclic graph

Kal

8:30 ↔ 9:00